



PREPARED BY

Muzaffar Makhkamov,
Uktamov Ibrohim,
Aziz Azimov,
Saidkomil Maxamadxo'jayev

December 10, 2024, Last edit April 20,, 2026

Partner

Tashkent

Documentation

The Unisoft Gate API facilitates seamless payment processing and balance transfers for Uzcard, Humo cards and  VISA direct (Beta version) .

Table of contents

A. Changes	6
A1. Upcoming Changes 	7
B. Terms and Definitions	7
C. Project Overview	8
C.1 General Scenario	8
C.3 Connection	9
3.3.1 Authorization	9
3.3.2 Generating Digest procedure explanation	10
C.4 Format	11
Request sample	11
Success—Sample Response	12
Error — Sample Response	12
D. Data types	13
E. Test	13
E.1 Cards	13
E.2 EPOS	14
1. Login	14
1.1 Login	14
Request	14
Response	15
1.2 Hello World	16
Request	16
1.3 Health Check	16
Request	16
2. Card	17
2.1 Card.register	17
Request	18
Sample responses	18
2.2 Card.verify	19
Request	19
Sample responses	20
2.3 Cards.get	21
Request	21
Sample response	21
 2.4 Card.remove	22
2.5 Card get by phone	22
Request	22
Sample response	23
2.6 Card.info	23
Request	23
Sample responses	24
2.7 Card Create	26
Request	26
Sample responses	28
2.8 Moment Card Register	30
Request	30
Sample responses	32
2.9 Pin set	33
Request	33

Sample responses	34
3.0 Pin Change	35
Request	35
Sample responses	35
3. Terminal	36
Terminal Types	37
3.1 Terminal add	37
Request	37
Sample responses	38
3.2 Terminal remove	38
Request	38
Sample responses	39
3.3 Terminal state	39
Request	39
Sample responses	40
4. Hold	41
4.1 Get OTP	41
Request	41
Sample responses	42
4.2 Create	44
Request	44
Sample responses	45
4.3 Confirm	45
Request	45
Sample responses	46
4.4 State	46
Request	46
Sample responses	46
4.5 Dismiss	47
Request	47
Sample responses	47
5. Payment	47
5.1 Payment using form	48
Payment Using Form Implementation Scenario	48
Request	49
Sample responses	50
5.2 Create	52
Request	52
Sample responses	53
5.2.1 Create without 3DS	55
Request	55
Sample responses	56
5.3 Confirm	58
Request	58
Sample responses	59
5.3.1 Confirm Wods	60
Request	60
Sample responses	60
5.4 Cancel	61
Request	61
Sample responses	62

5.4 State	63
Request	63
Sample responses	63
6. Transfer	63
6.1 Create	64
Request	64
Sample responses	65
6.2 Confirm	65
Request	65
Sample responses	65
6.3 Cancel	65
Request	65
Sample responses	66
6.4 State	66
Request	66
Sample responses	67
7. Account to Card	68
7.1 Create	68
Request	72
Sample responses	74
7.2 Confirm	75
Request	76
7.3 Cancel	76
Request	76
7.4 State	77
Request	77
8. Informational methods	79
8.1 Rate	79
Request	79
Sample responses	79
Response	80
8.2 Bin List	81
Request	81
Sample responses	81
Response	82
8.3 Balance	82
8.4 Registry	83
Account to Card file columns description	83
Payment registry file specs	84
8.5 Registry	86
Request	86
Sample responses	86
Response	86
Request	87
Request	88
10. Cabinet	89
10.1 How to access	89
States	89
Card states	89
Uzcard card statuses	89
Humo card statuses	90

Hold states	90
Payment states	91
Account To card states	91
Bank Codes	93
Error Codes	94
Basic error codes and their descriptions	94
Svgate error codes and their descriptions	95
Humo error codes and their descriptions	105
Tajikistan error codes and their description	111
Kyrgyzstan error codes and their description	111

A. Changes

Date	Version	Changes
15-May-2023	1.0	Methods: Hold , Payment , Account to Card
29-Nov-2024	1.0	Error codes from PC Svgate and Humo
3-Dec-2024	1.0	Added Hold methods Hold so changed methods' numbering
10-Dec-2024	2.0	Added card bind methods
11-Dec-2024	2.0	Added hash sum implementation
21-Dec-2024	2.0	- Updated method name payment.get.otp to payment.create - Added Registry file description and specs
25-Dec-2024	2.0	- Updated: Enhancements made to the Rate method. - Added: New method bin.list for retrieving BIN details. - Updated: access token description in login method
13-Jan-2025	2.0	- Updated: Test Unipos service scenarios enhanced - Updated: The Card management response format - Added: Terminal management (EPOS)
17-Jan-2025	2.0	Updated: Terminal management methods updated: <code>terminal.add</code>, <code>terminal.state</code>, <code>terminal.remove</code>
20-Jan-2025	2.0	- Updated: A2C cancellation procedure note - Added Hold state: 3 - "Hold successfully" - Added Payment, A2C state: -1 Accepted to Process
21-Jan-2025	2.0	- Added Hold methods - Numeration of methods has been changed 3-Terminal 4- Hold etc.

23-Jan-2025	2.0	Added Payment using form method
19-Feb-2025	2.0	- Updated info: Basic Error codes management scenario - Updated info: Card info results with response samples - Added VISA Direct for A2C: Uzcard/Humo not changed added VISA A2C option - Added Beta cabinet access realize - Added table to notify upcoming changes
20-Feb-2025	2.0	Added: A2C using phone number (by choosing bank code optional)
09-Apr-2025	2.0	- Updated: Payment states - Added: Card create method
11-Aug-2025	2.0	Added: <code>phone_number</code> param to <code>payment.create</code> method
06-oct-2025	2.0	Added: <code>register moment card, pin set, pin change</code> method
19-Aug-2025	2.0	Updated: 8.1 Rate -> Sample responses (More detailed description added) + <code>target_currency</code> field
20-Aug-2025	2.0	Added: Get card balance 2.3 cards.get via card token
20-Aug-2025	2.0	Added: get card numbers 2.5 card.get.by.phone via phone number
03-Oct-2025	2.0	Added: top up VISA card via phone number 7.0 Account to Card
28-Jan-2026	2.0	Fixed card.info method and added health.check method
20-Apr-2026	2.0	Fixed card.info method to get Visa card info by phone.

A1. Upcoming Changes

Modules	Description
Error Objects	Unique error management for all PC to ease error handling issues.
card info	Card info response unification
A2C	A2C RF cards (VISA, MASTER, MIR) addition.
Test Sceanrios	Test cases for enhanced testing methods.
Cabinet	Cabinet analytics

B. Terms and Definitions

Term	Definition
Client	Partner that allowed to use Unisoft Gate
User	Partners clients are preferred as User for Unisoft
Request	Request for some data
Response	Respond to the request
Database	Database which interacts with Gate
Registry	Registry service that registers every sensitivity data
Uzcard	Uzcard Online (Svgate)
Humo	Humo—processing centre owned by the Central Bank of Uzbekistan
Visa	Visa Processing centre
Transfer	Peer to Peer operation
Payment	Payment for services or debit operation in some cases
Exchange Conversion	Conversion of currency according to Universalbank JSC
CBU	Central Bank of Uzbekistan (in some reference used as CB)
Sender	Sender debit
Receiver	Receiver credit
Cheque	Cheque that made for a receiver who is waiting for payment from the sender
EPOS	Electron POS terminal which contains merchant_id/terminal_id

C. Project Overview

C.1 General Scenario

Test Scenario

Prerequisites

- **NDA:** Ensure a signed Non-Disclosure Agreement is in place.
- **IP Allowance:** Submit an IP allowance request via email. Once approved, your IP will be whitelisted.

Test Environment Setup

Test credentials and a Postman collection will be provided.

1. **Credentials:** Obtain the [.zip](#) folder containing credentials. Extract its content using the provided password.
2. **Digest Preparation:** Use the secret key to prepare a request digest, following the procedure detailed in [3.3.2 Generating Digest](#).
3. **Token Retrieval:** Authenticate by calling the [login](#) method to acquire a token.
4. **Service Test:** Test the service by sending a [hello.world](#) request. Include the token and the digest in the request.

Postman Collection Setup:

Open the Postman collection and update the following in Variables:

- username
- password
- secret

 Do not change other variables or scripts.

Send a request to the login method from your allowed IP.  If successful, you're ready to proceed!

Test Registry:

Test registry details will be sent to your email after test operations.

 All amounts in the test account will be **lowered to 100 (1 UZS) automatically** in the production environment you will get in both API and Registry actual amount

C.3 Connection

HOST	host/api/v1/jsonrpc
------	---------------------

3.3.1 Authorization

Authorization must be sent by the client in the header as:

Authorization: Bearer <code>{{access_token}}</code>	Access-Token will be returned by the login method
Content-type: Application/json	Gate serves for only JSON request

Accept: Application/json	If the client sends a valid request, Gate response will be in JSON format
Header-Login	Partners username
Header-Sign	digest 3.3.2 —hash of request body

3.3.2 Generating Digest procedure explanation

1. `secret`: The secret variable represents the secret key used for HMAC calculation. It should be kept confidential and securely managed to ensure the security of the authentication process.

```
secret = b"secret_must_be_saved_secretly_!!!"
```

2. **HMAC Digest Calculation**: The `hmac.new()` function computes the HMAC digest using the provided secret key (`secret`), the request body (`request.body`), and the SHA-256 hash function (`hashlib.sha256`). This process generates a unique digest based on the combination of the secret key and the message content.

```
hmac_digest = hmac.new(secret, request.body, hashlib.sha256)
```

3. **Base64 Encoding**: The resulting digest is then encoded in base64 format using `base64.b64encode()` to ensure compatibility with various data transmission protocols and systems.

```
digest_base64 = base64.b64encode(hmac_digest.digest()).decode()
```

4. **Set digest to Header of Request**: Finally, the calculated digest (`digest_base64`) must be added to the header as the key `<< Header-Sign: {{digest_base64}} >>`. Ensure that the secret key is securely managed and not exposed to unauthorized parties. Additionally, consider integrating this HMAC digest calculation mechanism into your service for secure message authentication and integrity verification.

Additional

1. Postman Pre-request code snippet

```
crypto = require('crypto-js')  
  
// Retrieve the secret from the environment
```

```
var secret = 'secret_very_very!';

var resolvedBody = pm.request.body.raw;

// Generate the HMAC using SHA-256

var encrypted = crypto.HmacSHA256(resolvedBody, secret);

var digest = crypto.enc.Base64.stringify(encrypted);

// Set the digest as an environment variable (or another collection variable if
needed)

pm.collectionVariables.set('digest', digest);
```

2. The final request code snippet should be as below:

```
curl --location 'https://{{host}}/api/v1/jsonrpc' \
--header 'Header-Login: {{username}}' \
--header 'Header-Sign: {{digest}}' \
--header 'Content-Type: application/json' \
--header 'Authorization: Bearer {{access_token}}' \
--data '{
  "jsonrpc": "2.0",
  "id": 111,
  "method": "transfer.create",
  "params": {
    "ext_id": "12",
    "number": "8600xxxx",
    "amount": 100000,
    "currency": 643
  }
}'
```

C.4 Format

[Jsonrpc 2.0](#)

Request sample

POST
{

```

    "jsonrpc": "2.0",
    "id": "1",
    "method": "name",
    "params": {
        "key": "value"
    }
}

```

jsonrpc	String	2.0
id	String or numeric	Request id (preferred uuid or client name + time: company_12345)
method	String	Method name
params	Array	Required named params as method structure

Success—Sample Response

```

{
    "jsonrpc": "2.0",
    "id": "1",
    "status": true,
    "origin": "login",
    "result": {
        ~
        "message": { ~ }
    },
    "host": {
        "host": "visa.unired.uz",
        "time_stamp": "2020-01-20 11:59:59"
    }
}

```

Comments

id	String	Request id emitted
status	boolean	true in success response
origin	String	Origin of response in Gate
result	Object	Required params as method structure
message	Object	Result message
host	Object	Info about host

Error — Sample Response

```
{
  "jsonrpc": "2.0",
  "status": false,
  "origin": "login",
  "error": {
    "code": 401,
    "message": { ~ }
  },
  "id": null,
  "host": {
    "host": "visa.unired.uz",
    "time_stamp": "2020-01-20 11:59:59"
  }
}
```

Comments

id	String	null
status	boolean	false in error response
origin	String	Origin of response in Gate
error	Object	Required params as method structure
code	Integer	Error code
message	Object	Error message
host	Object	Info about host

D. Data types

Name	Type	Description
number	String	Plain card number 16 characters.
amount	Integer	Amount must not contain any float or decimal points
message	Object/String	
host	Object	
error	Object	
result	Object	
code	Integer	

E. Test

E.1 Cards

PC	Card Number	Expire	CVC		Owner
UZCARD	8600 4829 5296 4119	0426	-	+998 99 976 04 30	KHUSANBOY ABDULKODIR
HUMO	9860 2301 0122 3272	0426	-	+998 99 976 04 30	KHUSANBOY ABDULKODIR
VISA	4934 5377 7076 7402	0426	-	-	SANJAR MAKHMUDJANOV
VISA (UZS)					

E.2 EPOS

	PC	Merchant ID	Terminal ID	Type	Name
Payment	Uzcard	90489288	92431998	1	Universalbank
	Humo	01186000011860G	2366191P	1	Universalbank
A2C	Uzcard	90489288	92414051	3	Universalbank
	Humo	01186000011860G	2366191P	3	Universalbank
	VISA	29896840200001186064		5	Universalbank

Phone Number	Owner
+998999760430	Muzaffar Makhkamov

1. Login

1.1 Login

Login method used to get access token

Request

username	String	Client login
----------	--------	--------------

password	String	Client password
----------	--------	-----------------

POST	
	<pre>{ "jsonrpc": "2.0", "id": "{{randomInt}}", "method": "login", "params": { "username": "{{username}}", "password": "{{password}}" } }</pre>

Response

access_token	String	access_token - The access_token is used to authorize other requests. Default Validity: The access token is valid for 60 minutes by default. Extended Validity: If using IPsec, you can request a 365-day validity by contacting your project curator at Unisoft for configuration.
--------------	--------	---

<pre>{ "jsonrpc": "2.0", "result": { "access_token": "secret_lifo_token*****" }, "id": "976", "status": true, "origin": "login", "host": { "host": "unipos.online", "timestamp": "2023-04-18 09:24:05.834127" } }</pre>	Comments
<pre>{ "jsonrpc": "2.0", "error": { "code": -32101, "message": { "uz": "Foydalanuvchi nomi yoki parol noto'g'ri", "ru": "Имя пользователя или пароль неверен", "en": "Username or password incorrect" } }, "data": null, "id": "489", "status": false, "origin": "login", "host": {</pre>	

```

    "host": "unipos.online",
    "timestamp": "2023-04-18 09:46:29.363561"
  }
}

```

1.2 Hello World

Hello world is a test method for checking authorization is ok and a hash of the request

Request

input	String	Any valid string
-------	--------	------------------

POST	
<pre> { "jsonrpc": "2.0", "id": "{{\${randomInt}}}", "method": "hello.world", "params": { "input": "any_valid_string" } } </pre>	
<pre> { "jsonrpc": "2.0", "result": { "message": "any_valid_string" }, "id": 213232, "status": true, "origin": "hello.world", "host": { "host": "Unipos", "timestamp": "2024-12-21 11:18:25.579212" } } </pre>	

1.3 Health Check

The health.check method is used to verify that the service is running and accessible. It can be used for basic availability checks, monitoring, and load balancer health validation.

Request

POST	
<pre> { "jsonrpc": "2.0", "id": {{\${randomInt}}}, "method": "health.check", "params": { } } </pre>	


```
{
  "jsonrpc": "2.0",
  "result": {
    "status": "UP",
    "db": {
      "status": "OK",
      "elapsed_time": "4e-06s"
    },
    "app": {
      "status": "OK",
      "elapsed_time": "2.5e-05s"
    }
  },
  "id": 1,
  "status": true,
  "origin": "health.check",
  "host": {
    "host": "Unipos",
    "timestamp": "2026-01-28 11:01:35.639527"
  }
}
```

2. Card

Card Management Implementation Scenario:

Method for card registration, OTP verification, and binding to a client's phone number.

1. Register a Card and Request OTP:
 - 1.1. Use card.register to initiate the card registration process and request an OTP code. The OTP will be sent to the client's linked card phone number.
2. Verify Card:
 - 2.1. Use card.verify with the ext_id and OTP code to validate the card and bind it to the client's phone number. If the OTP code is correct, the card is successfully verified.
 - 2.2. Upon successful verification, retrieve and save the token for further operations.
3. Perform Operations with Token:
 - 3.1. Use the saved card token for further operations such as:
 - 3.1.1. card.details (to check balance, status, payment and monitoring).
 - 3.1.2. transfer (for p2p transfers).
 - 3.2. Avoid using plain card data—only use the card token for security and privacy.
4. Deactivate Card Token (if not in use):
 - 4.1. If the card token is no longer needed, ensure it is deactivated to prevent unauthorized access and ensure security.



2. Card

2.1 Card.register

Method for binding card to the system to get card token

Through this method, the card is registered using OTP, and you are provided with a token to work with card information.

The method allows sending an OTP to the phone number registered to the card.

The OTP usage time is no more than 2 minutes.

Only one token can be valid for a single card at a time. If the card expiration date is entered incorrectly three times in a row, the card specified in the request will be blocked for one day.

Request

jsonrpc	String	"2.0"
method	String	card.register
id	String Integer	Request Id
params	Object	
number	String	Plain card number 16 characters.
expire	String	Plain cards expire 4 characters in format "YYmm" eg: 2701.
phone	Optional	phone number registered to the card

POST

```
{
  "jsonrpc": "2.0",
  "id": "{{randomInt}}",
  "method": "card.register",
  "params": {
    "number": "{{number}}",
    "expire": "{{expire}}",
    "phone": {{phone}}
  }
}
```

Sample responses

ext_id	Integer	
--------	---------	--

phone_mask	String	
mask	String	
verified	Boolean	
pc_type	Integer	

```
{
  "jsonrpc": "2.0",
  "result": {
    "ext_id": "e8f589ea-8ce6-414c-90ee-bc852b7428de",
    "phone_mask": "*****2310",
    "mask": "561468*****1746",
    "verified": false,
    "pc_type": 1
  },
  "id": "31321",
  "status": true,
  "origin": "card.register",
  "host": {
    "host": "Unipos",
    "timestamp": "2026-04-01 10:31:24.565734"
  }
}
```

2.2 Card.verify

Method to confirm OTP and receive a real token. Tokenization involves replacing the card numbers with a randomly generated combination of numbers and letters, which will be used to replace the main bank card number.

Note:

1. Only one token can be valid for one card at a time.
2. After 3 incorrect confirmation attempts, registration will be blocked for 24 hours.

Request

jsonrpc	String	"2.0"
method	String	card.verify
id	String Integer	Request id
params	Object	



ext_id	String	External id of operation must be unique for Partner
code	Integer	sms code

POST

```
{
  "jsonrpc": "2.0",
  "id": "{{randomInt}}",
  "method": "card.verify",
  "params": {
    "ext_id": "{{ext_id}}"
    "code": "{{OTP_client_code}}"
  }
}
```

Sample responses

ext_id	String	
phone_mask	String	
mask	String	
token	Integer	
pc_type	Integer	

```
{
  "jsonrpc": "2.0",
  "result": {
    "ext_id": "474c3dff8e10478ab99a442089cf57e1",
    "phone_mask": "+998*****00",
    "mask": "986017*****6810",
    "token": "1133B3006ACC4142A6ED8DD9A80493D7",
    "pc_type": 3
  },
  "id": "31321",
  "status": true,
  "origin": "card.verify",
  "host": {
    "host": "Unipos",
    "timestamp": "2026-04-01 10:34:46.734604"
  }
}
```

2.3 Cards.get

This method allows getting detailed information about the card using a token. If necessary, information for multiple cards can be requested at the same time by specifying several tokens in the request.

Request

jsonrpc	String	"2.0"
method	String	cards.get
id	String Integer	Request Id
params	Object	
tokens	String[List]	Card tokens are acquired using card.verify

POST

```
{
  "jsonrpc": "2.0",
  "id": "${randomInt}",
  "method": "cards.get",
  "params": {
    "tokens": [
      {
        "token": "8F481DD8B5C6*****",
        "pcy_type": 1
      }
    ]
  }
}
```

Sample response

```
{
  "jsonrpc": "2.0",
  "result": [
    {
      "uzcard": [
        {
          "id": "9CD94141BC484EE1C23D65284C",
          "mask": "860004*****6858",
          "status": 0,
          "phone": "9989***7910",
          "full_name": "ISROILOV SUXROBJON",
          "balance": 11119,
          "sms": false,
          "card_type": "private"
        }
      ]
    }
  ],
  "id": "31321",
  "status": true,
}
```

```
"origin": "cards.get",
"host": {
  "host": "Unipos",
  "timestamp": "2025-08-20 17:26:26.227033"
}
}
```



2.4 Card.remove



Method is in the development stage and will be released soon!

The method allows deleting a partner's token from UNIPOS using a temporary token.

2.5 Card get by phone

The method allows retrieving a list of cards linked to a phone number.

Request

jsonrpc	String	"2.0"
method	String	card.get.by.phone
id	String Integer	Request Id
params	Object	
phone	String	Phone number

POST

```
{
  "jsonrpc": "2.0",
  "id": "{{randomInt}}",
  "method": "card.get.by.phone",
  "params": {
    "phone": "99893598****"
  }
}
```

Sample response

```
{
  "jsonrpc": "2.0",
  "result": [
    {
      "card_number":
"5A78BC1140B64001AA3C8131962BD540",
      "card_mask": "561468*****4142",
      "full_name": "ABDULLAJON MAXMUDOV",
      "bank_name": "Universal bank",
    }
  ]
}
```

```

        "pc_type": "uzcard"
    },
    {
        "card_number":
"D4D1BBDD45A84BFA2FCFE419744A5F37",
        "card_mask": "561468*****4290",
        "full_name": "ABDULLAJON MAXMUDOV",
        "bank_name": "Universal bank",
        "pc_type": "uzcard"
    }
],
    "id": "31321",
    "status": true,
    "origin": "card.get.by.phone",
    "host": {
        "host": "Unipos",
        "timestamp": "2025-08-21 10:34:36.003062"
    }
}

```

2.6 Card.info

The method for retrieving data. In this method, when the recipient's card number is provided, the following data is returned in the response: **full name, processing centre type, issuing bank name and card status.**

Request

jsonrpc	String	"2.0"
method	String	card.info
id	String Integer	Request Id
params	Object	
number	String	Plain card number 16 characters or Phone number.
pc_type	Integer	pc_type = 5 indicates that the provided card number belongs to the VISA payment system and the request will be processed as a VISA card.

POST

```

{
    "jsonrpc": "2.0",
    "id": "{{ $randomInt }}",
    "method": "card.info",

```

POST

```
{
  "params": {
    "number": "{{number}}",
    "pc_type": "{{pc_type}}",
  }
}
```

Sample responses

is_credit	Boolean	Indicates whether the card is a credit card or not
card_number	String	
owner	String	Card holder name
bank	String	Bank name
state	Integer	State of the card
phone	String (Nullable)	the phone number to which the card is linked
pc_type	Integer	Processing centre id in Unipos Gateway 1- for Uzcard 3- for Humo card 5- VISA card 7- Tajikistan Korti Milli 8- Kyrgyzstan Mbank and ElKart

```
{
  "jsonrpc": "2.0",
  "result": {
    "card_number": "9860****6810",
    "owner": "IBROKHIM UKTAMOV",
    "state": 0,
    "bank": "IpakYuli",
    "pc_type": 3,
    "is_corporate": false
  },
  "id": 1,
  "status": true,
  "origin": "card.info",
  "host": {
    "host": "Unipos",
    "timestamp": "2026-01-28 10:56:04.697194"
  }
}
```

HUMO

```
{
  "jsonrpc": "2.0",
  "result": {
```

UZCARD


```

        "card_number": "561468*****4552",
        "owner": "OKTAMOV IBROXIM OYBEK OGLI",
        "state": 0,
        "bank": null,
        "pc_type": 1,
        "is_corporate": false
    },
    "id": 1,
    "status": true,
    "origin": "card.info",
    "host": {
        "host": "Unipos",
        "timestamp": "2026-01-28 10:54:10.303964"
    }
}

```

```

{
    "jsonrpc": "2.0",
    "result": {
        "card_number": "493453*****3509",
        "owner": "SANJARBEK MAXMUDJANOV",
        "state": 0,
        "bank": "JSCB Universalbank",
        "pc_type": 5,
        "is_corporate": false,
        "currency": "840"
    },
    "id": 1,
    "status": true,
    "origin": "card.info",
    "host": {
        "host": "Unipos",
        "timestamp": "2025-02-19 16:15:21.448608"
    }
}

```

VISA

Limit card :
min: 0.01 dollar
max: 1500 dollar

```

{
    "jsonrpc": "2.0",
    "result": {
        "card_number": "505827*****0789",
        "owner": null,
        "state": 0,
        "bank": "ЗАО «Душанбе Сити Банк»",
        "pc_type": 7,
        "is_corporate": false,
        "limit": {
            "min": 100000,
            "max": 1500000000
        },
        "logo": {
            "bank_logo": "https://cdn-api.cloudgate.u.pngg",
            "pc_logo": "https://cdn-api.cloudgate.u.png"
        }
    },
    "id": 1,
    "status": true,
    "origin": "card.info",
    "host": {
        "host": "Unipos",

```

Korti Milli

Limit card :
min: 1 000 UZS
max: 15 000 000 UZS

```

    "timestamp": "2026-01-28 10:50:58.462633"
  }
}

```

```

{
  "jsonrpc": "2.0",
  "result": {
    "card_number": "471470*****2501",
    "owner": "AKBARALI TULKINOV",
    "state": 0,
    "bank": "Mbank",
    "pc_type": 8,
    "is_corporate": false
  },
  "id": 1,
  "status": true,
  "origin": "card.info",
  "host": {
    "host": "Unipos",
    "timestamp": "2026-01-28 11:00:26.749558"
  }
}

```

KYRGYZSTAN

Limit card :
min: 5000 UZS
max: 20 000 000 UZS

2.7 Card Create

The method is used to create a virtual card for the client (currently **Uzcard**; later **Humo** and **Visa virtual** cards will be supported).

Requirements

1. Client data validation: All personal data for the client must be retrieved from the **MYID** service to ensure accuracy and compliance.
2. **SMS Confirmation (2FA)**: Before calling this method, an **SMS confirmation must be successfully completed** by the client. Only after confirmation should the card be created.

Request

jsonrpc	String	"2.0"
method	String	card.create
id	String Integer	Request Id
params	Object	
ext_id	String	External unique operation ID (UUID format recommended)
processing	String	Specifies the processing center (e.g., svgate for Uzcard)

card_type	String	Type of the card to be created (only "virtual" supported)
product_type	String	Internal product type code defined by the bank
client	Object	retrieved data from the MYID service
inn	String	
pnfl	String	
last_name	String	
first_name	String	
middle_name	String	
birth_date	String	
birth_place	String	
gender	String	
doc_type	String	
series	String	
number	String	
doc_issue_date	String	
residence_region	String	
residence_district	String	
residence_address	String	
phone	String	
birth_country	String	
citizenship	String	
residence_country	String	
email	String	
marital_status	String	
code_filial	String	01186 - Universalbank Digital Branch code

POST

```
{
  "id": 0,
  "jsonrpc": "2.0",
  "method": "card.create",
  "params": {
    "ext_id": "{{randomUUID}}",
    "processing": "svgate",
    "card_type": "virtual",
    "product_type": "01",
    "client": {
      "inn": "000000000",
      "pnfl": "501020657300",
      "last_name": "RAVSHANOV",
      "first_name": "SHAHZODBEK",
      "middle_name": "SHUHRAT",
      "birth_date": "01.02.2006",
      "birth_place": "YAKKABOG' TUMANI",
      "gender": "1",
      "doc_type": "0",
      "series": "AD",
      "number": "4204421",
      "doc_issue_date": "07.08.2023",
      "doc_expire_date": "06.08.2033",
      "doc_issue_place": "YAKKABAGSKIY ROVD KASHKADARINSKOY OBLASTI",
      "residence_region": "10",
      "residence_district": "051",
      "residence_address": "Bynyodkor MFY, Tyrkiston kychasi, 31-yy",
      "phone": "998976782442",
      "birth_country": "182",
      "citizenship": "182",
      "residence_country": "182",
      "email": "email",
      "marital_status": "1",
      "code_filial": "01186"
    }
  }
}
```

Sample responses

card_number	string	The full number of the newly issued virtual card
expire		Expiration date of the card in MM/YY format
embossed_name		Name printed on the card (usually client's middle or full name)
account		Card account number linked to the virtual card

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
    "card_number": "8600123412341234",
    "expire": "12/27",
    "embossed_name": "SHUHRAT RAVSHANOV",
  }
}
```

```
}    "account": "12345678901234567890123456"  
}
```

2.8 Moment Card Register

The method is used for registering Uzcard Moment cards for customers..

Requirements

1. The **moment.card.create** method is used to send the client's passport information and the card number to be registered
2. After the card is registered, its status can be retrieved using the **card.info** method.
If **state = 14** is returned, it means the card has been successfully registered.
The card registration process takes approximately **5 to 10 minutes**.
3. The card is registered in **SVGATE** and a token is obtained using the **card.register** and **card.verify** methods. In the **card.register** method, the **phone** key is mandatory.
4. The **moment.pin.set** method is used to set a PIN code for the card.
The **pin.set** request can only be sent for cards with **state = 14**.
5. After the PIN is set, the card's status changes to **state = 0**, and the card becomes active.
6. The **moment.pin.change** method is used to change the previously set PIN code.

Request

jsonrpc	String	"2.0"
method	String	moment.card.create
id	String Integer	Request Id
params	Object	Contains the main parameters required for the request.
number	String	card number
client	Object	retrieved data from the MYID service
inn	String	Client's Individual Taxpayer Identification Number (INN)
pnfl	String	Client's Personal Identification Number (PNFL)
last_name	String	Client's last name (surname).
first_name	String	Client's first name (given name).
middle_name	String	Client's middle name or patronymic, if available.

birth_date	String	Date of birth of the client (format: YYYY-MM-DD).
birth_place	String	Place of birth of the client
gender	String	Gender of the client (e.g., "M" for male, "F" for female)
doc_type	String	Type of identification document (e.g., passport, ID card)
series	String	Series (prefix) of the document
number	String	Document number
doc_issue_date	String	The date when the document was issued
residence_region	String	Client's region of residence.
residence_district	String	Client's district (area) of residence
residence_address	String	Full residential address of the client
phone	String	Client's phone number
birth_country	String	The country where the client was born
citizenship	String	The client's citizenship or nationality
residence_country	String	The country where the client currently resides
email	String	Client's email address
marital_status	String	Client's marital status (e.g., single, married)
code_filial	String	01186 - Universalbank Digital Branch code
doc_expire_date	String	The expiration date of the identification document
mobile_phone	String	Client's mobile phone number
region	String	The region code or name related to the client's location

POST

```
{
  "params": {
    "number": "5614681079954552",
    "client": {
      "ext_id": "{{ $randomUUID }}",
      "inn": "000000000",
      "pnfl": "33008*****0026",
      "last_name": "UKTAMOV",
      "first_name": "IBROHIM",
      "middle_name": "OYBEK O'G'LI",

```

POST

```

    "birth_date": "30.08.1999",
    "birth_place": "CHORTOQ TUMANI",
    "birth_country": "860",
    "gender": "1",
    "citizenship": "860",
    "doc_type": "6",
    "series": "AB",
    "number": "4***384",
    "doc_issue_date": "28.06.2016",
    "doc_expire_date": "27.06.2026",
    "doc_issue_place": "NAMANGAN VILOYATI CHORTOQ IIB",
    "residence_country": "860",
    "residence_region": "456",
    "residence_district": "45",
    "residence_address": "CHORTOQ TUMANI",
    "phone": "99893598**00",
    "mobile_phone": "99893598**00",
    "email": "email",
    "marital_status": "1",
    "code_filial": "01186",
    "region": "CHORTOQ"
  }
},
"id": 0,
"jsonrpc": "2.0",
"method": "moment.card.create"
}

```

Sample responses

card_number	string	card number
phone		Client's phone number
description		Description

```

{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
    "card_number": "561468107***4552",
    "expire": "12/27",
    "description": "registered"
  }
}

```


2.9 Pin set

This method is used to set a new **PIN code** for the customer's card.

Once the PIN is successfully set, the card becomes **active (state = 0)**

Request

jsonrpc	String	"2.0"
method	String	moment.pin.set
id	String Integer	Request Id
params	Object	
number	String	Plain 16-digit card number for which the PIN will be set..
merchant	String	Merchant identifier initiating the request.
terminal	String	Terminal identifier from which the request is sent.
ext_id	String	External unique identifier of the request, usually generated by the partner system.
redirect_url	String	Optional redirect or callback URL for response delivery.

POST

```
{
  "jsonrpc": "2.0",
  "id": "12",
  "method": "moment.pin.set",
  "params": {
    "card_id": "CA9950A58AC3F0F***30811AC602D",
    "merchant": "90**697",
    "terminal": "924**618",
    "ext_id": "{{${randomUUID}}}",
    "redirect_url": "https://online.unired.uz/fallback/merchant?categoryId=1"
  }
}
```



Sample responses

ext_id	String	external id
url	String	redirect url

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
    "ext": "test_48",
    "url": "https://webapi.uzcard.uz:27007/pin/pi*****"
  },
}
```

```
}

```

3.0 Pin Change

This method is used to change the **PIN** code for the customer's card

Request

jsonrpc	String	"2.0"
method	String	moment.pin.change
id	String Integer	Request Id
params	Object	
number	String	Plain 16-digit card number for which the PIN will be set..
merchant	String	Merchant identifier initiating the request.
terminal	String	Terminal identifier from which the request is sent.
ext_id	String	External unique identifier of the request, usually generated by the partner system.
redirect_url	String	Optional redirect or callback URL for response delivery.

POST

```
{
  "jsonrpc": "2.0",
  "id": "12",
  "method": "moment.pin.change",
  "params": {
    "card_id": "CA9950A58AC3F0F***30811AC602D",
    "merchant": "90**697",
    "terminal": "924**618",
    "ext_id": "{{ $randomUUID }}",
    "redirect_url": "https://online.unired.uz/fallback/merchant?categoryId=1"
  }
}
```

Sample responses

ext_id	String	External unique identifier of the request, usually generated by the partner system.
--------	--------	---

url	String	Redirect URL where the form is opened
-----	--------	---------------------------------------

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
    "ext": "test_48",
    "url": "https://webapi.uzcard.uz:27007/pin/pi*****"
  },
}
```

3. Terminal

E-POS is an online POS system for handling operations in Uzcard (SV-Gate), Humo, and VISA. In VISA, the account number serves the same role as an E-POS.

E-POS consists of two key components:

- merchant_id
- terminal_id

This setup is ideal for partners with multiple E-POS terminals, such as:

- Multiple Locations: E.g., three stores, each with its own E-POS.
- Different Services: E.g., mobile app, website, etc.
- Sub-Agents: E.g., hotel networks managing points individually.

 For Single E-POS Users:

- Partners with only one E-POS point will have it added by default for Uzcard and Humo, as requested by the coordinator.
- In such cases, merchant_id and terminal_id are optional, as the system retrieves the E-POS automatically from the registry and sends it to PCs (SV-Gate and Humo).

Terminal (EPOS) management implementation:

1. Get E-POS Information:
 - 1.1. Retrieve your E-POS details from the back office or your coordinator.
2. Register E-POS:
 - 2.1. Send a request to terminal.add to register the E-POS with the desired payment type (1: Payment) :
3. Check Terminal State:
 - 3.1. After registration, verify the terminal status using the terminal.state method.
4. Remove Terminal:
 - 4.1. If needed, send a request to terminal.remove to deactivate the terminal.
5. Manage Terminals:
 - 5.1. Check and manage your terminals via the back office or cabinet.

Terminal Types

Value	Description
1	Payment (charging money from a card hold_payment)
2	Credit Transfer (sending money to a card account to card A2C)
3	Transfer (P2P operations transfers)

3.1 Terminal add

Request

jsonrpc	String	"2.0"
method	String	terminal.add
id	String Integer	Request id
params	Object	
terminal	String	Terminal Identifier
merchant	String	Merchant Identifier
type	Integer	Terminal type
name	String	Terminal name with concise meaning
purpose	String	Purpose of terminal
account	String	Back office or coordinator provider account number for this EPOS

POST

```
{
  "jsonrpc": "2.0",
  "id": "123",
  "method": "terminal.add",
  "params": {
    "merchant": "{{merchant_humo}}",
    "terminal": "{{terminal_humo}}",
    "type": 1,
    "name": "Test LTD Mobile app",
    "purpose": "Payment for services local",
    "account": "202010203040506070809"
  }
}
```



Sample responses

merchant	String	Merchant Identifier
terminal	Object	Terminal Identifier
type	Integer	Terminal type
name	String	Symbolic Name
purpose	String	Purpose
is_active	Boolean	Whether the terminal is active or not

```
{
  "jsonrpc": "2.0",
  "result": {
    "merchant": "{{merchant_id}}",
    "terminal": "{{terminal_id}}",
    "type": {{terminal_type}},
    "name": "{{terminal_name}}",
    "purpose": "{{terminal_purpose}}",
    "is_active": true
  },
  "id": "372",
  "status": true,
  "origin": "terminal",
  "host": {
    "host": "unipos.online",
    "timestamp": "2024-12-03 05:31:18.491895"
  }
}
```

3.2 Terminal remove

Request

jsonrpc	String	"2.0"
method	String	terminal.remove
id	String Integer	Request id
params	Object	
terminal	String	Terminal Identifier
merchant	String	Merchant Identifier

POST

```
{
  "jsonrpc": "2.0",
```



POST

```
{
  "id": "123",
  "method": "terminal.remove",
  "params": {
    "merchant": "{{merchant_humo}}",
    "terminal": "{{terminal_humo}}"
  }
}
```

Sample responses

merchant	String	Merchant Identifier
terminal	Object	Terminal Identifier
type	Integer	Terminal type
name	String	Symbolic Name
purpose	String	Purpose
is_active	Boolean	Whether the terminal is active or not

```
{
  "jsonrpc": "2.0",
  "result": {
    "merchant": "{{merchant_id}}",
    "terminal": "{{terminal_id}}",
    "type": {{terminal_type}},
    "name": "{{terminal_name}}",
    "purpose": "{{terminal_purpose}}",
    "is_active": false
  },
  "id": "372",
  "status": true,
  "origin": "terminal",
  "host": {
    "host": "unipos.online",
    "timestamp": "2024-12-03 05:31:18.491895"
  }
}
```

3.3 Terminal state

Request

jsonrpc	String	"2.0"
method	String	terminal.state
id	String Integer	Request id
params	Object	



terminal	String	Terminal Identifier
merchant	String	Merchant Identifier

POST

```
{
  "jsonrpc": "2.0",
  "id": "123",
  "method": "terminal.state",
  "params": {
    "merchant": "{{merchant_humo}}",
    "terminal": "{{terminal_humo}}"
  }
}
```

Sample responses

merchant	String	Merchant Identifier
terminal	Object	Terminal Identifier
type	Integer	Terminal type
name	String	Symbolic Name
purpose	String	Purpose
is_active	Boolean	Whether the terminal is active or not

```
{
  "jsonrpc": "2.0",
  "result": {
    "merchant": "{{merchant_id}}",
    "terminal": "{{terminal_id}}",
    "type": {{terminal_type}},
    "name": "{{terminal_name}}",
    "purpose": "{{terminal_purpose}}",
    "is_active": true
  },
  "id": "372",
  "status": true,
  "origin": "terminal",
  "host": {
    "host": "unipos.online",
    "timestamp": "2024-12-03 05:31:18.491895"
  }
}
```

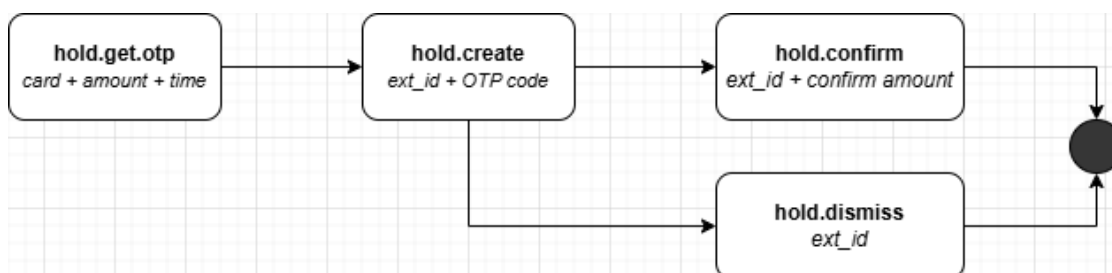

4. Hold

Temporarily hold an amount on the client's card for up to 60 days (maximum: 86,400 minutes).

Note: If the partner does not process the hold.confirm method to complete the charge within this period, the held amount will be automatically refunded to the client's card.

Hold Implementation Scenario:

1. Get OTP:
 - 1.1. Use hold.get.otp to request an OTP code, specifying the hold duration.
2. Check Hold State:
 - 2.1. If the method returns "hold": {state: -2}, call hold.create with the external ID (ext_id) and the OTP code to initiate the hold.
3. Confirm Hold:
 - 3.1. If the OTP code is valid, use hold.confirm to finalize the charge, transferring the held amount to the partner's account.



4.1 Get OTP

Method creates hold and sends to client sms OTP code to bound card

Request

jsonrpc	String	"2.0"
method	String	hold.get.otp
id	String Integer	Request id
params	Object	
ext_id	String	External id of operation must be unique for Partner
number	String	Plain card number 16 characters.
expire	String	Plain card expiry 4 characters in format "YYmm" e.g.: 2701.
receiver_id	String	Receiver id such as card number, account, etc.

		Note: The unique client code (receiver_id) —the end recipient of the funds, which allows identification within the framework of inter-system interaction. For example, 1. For mobile number payment—the full phone number; 2. For payment to an e-wallet—the full e-wallet number 3. For currency exchange operations: PINFL or the user's phone number, etc. This parameter is not processed or analyzed but is required to meet legal requirements for combating money laundering (AML).
amount	Integer	Amount must be evaluated in tiyins (UZS * 100)
time	Integer	Hold time integer in minutes after expiry of time hold amount will be refunded to the client
merchant_id	String	Merchant ID
terminal_id	String	Terminal ID

POST	
	<pre>{ "jsonrpc": "2.0", "id": "{{ \$randomInt }}", "method": "hold.get.otp", "params": { "ext_id": "{{ unique_ext_id }}", "number": "{{ card_number }}", "expire": "2709", "receiver_id": "{{ receiver_id }}", "amount": 100, "time": 60, "merchant_id": "{{ merchant_id }}", "terminal_id": "{{ terminal_id }}" } }</pre>

Sample responses

ext_id	String	External id of operation must be unique for Partner
hold	Object	
state	Integer	State of payment
number	String	Card number
description	String	Description of payment

amount	Integer	Amount in tiyins (UZS * 100)
currency	String	Currency code, 860=UZS
commission	Integer	Commission
account	List	
card	Object	
mask	String	
owner	Object	
phone	Object	
payment	Object	
ref_num	String	
id	Integer	
merchant	Object	
organization	String	
epos	Object	
type	Object	

```
{
  "jsonrpc": "2.0",
  "result": {
    "ext_id": "test_ext_1737614192723",
    "hold": {
      "state": -2,
      "description": "hold pending",
      "amount": 100
    },
    "currency": "860",
    "commission": 0,
    "account": [
      {
        "name": "card",
        "title": {
          "ru": "Карта для списания",
          "en": "Paying Card",
          "uz": "Mablag' yuboruvchi karta"
        },
        "mask": "860048*****4119"
      },
      {
        "name": "owner",
        "title": {
          "ru": "Плательщик",
          "en": "Payer",
          "uz": "To'lovchi"
        }
      }
    ]
  }
}
```

```

    },
    "value": "ABDUL QODIR HUSANBOY"
  },
  {
    "name": "phone",
    "title": {
      "ru": "Номер телефона",
      "en": "Phone number",
      "uz": "Telefon raqami"
    },
    "mask": "*****0430"
  },
],
"payment": {
  "ref_num": "HLD-TT-P-012325063828-fbc74a3b-a560-4268",
  "id": null
},
"merchant": {
  "organization": "Unisoft",
  "epos": {
    "merchant": "90489288",
    "terminal": "92431998"
  },
  "type": {
    "ru": "Оплата",
    "en": "Payment",
    "uz": "To'lov"
  }
},
},
"id": "123",
"status": true,
"origin": "hold.get.otp",
"host": {
  "host": "Unipos",
  "timestamp": "2025-01-23 06:38:29.249581"
}
}

```

4.2 Create

Method creates hold cheque using OTP code from the client

Request

jsonrpc	String	"2.0"
method	String	hold.create
id	String Integer	Request id
params	Object	
ext_id	String	External id of operation must be unique for Partner
code	String	OTP code which is sent to the client

POST

```
{
  "jsonrpc": "2.0",
  "method": "hold.create",
  "id": "{{randomInt}}",
  "params": {
    "ext_id": "{{unique_ext_id}}",
    "code": "{{OTP_CODE_CLIENT}}"
  }
}
```

Sample responses

i Response hold.confirm method same as hold.get.otp response only differs with state and payment id, RRN

4.3 Confirm

Method to confirm hold amount

Request

jsonrpc	String	"2.0"
method	String	hold.confirm
id	String Integer	Request id
params	Object	
ext_id	String	External id of operation must be unique for Partner
confirm_amount	Integer	The amount may vary from the initial value provided in hold.get.otp

POST

```
{
  "jsonrpc": "2.0",
  "method": "hold.confirm",
  "id": "{{randomInt}}",
  "params": {
    "ext_id": "{{unique_ext_id}}",
    "confirm_amount": 1000
  }
}
```

POST	
	<pre> } } </pre>

Sample responses

i Response hold.confirm method same as hold.get.otp response only differs with state and payment id, RRN

4.4 State

Method to confirm hold amount

Request

jsonrpc	String	"2.0"
method	String	hold.confirm
id	String Integer	Request id
params	Object	
ext_id	String	External id of operation must be unique for Partner
confirm_amount	Integer	The amount may vary from the initial value provided in hold.get.otp

POST	
	<pre> { "jsonrpc": "2.0", "method": "hold.state", "id": {{randomInt}}, "params": { "ext_id": "{{unique_ext_id}}" } } </pre>

Sample responses

i Response hold.confirm method same as hold.get.otp response only differs with state and payment id, RRN

4.5 Dismiss

Method to confirm hold amount

Request

jsonrpc	String	"2.0"
method	String	hold.dismiss
id	String Integer	Request id
params	Object	
ext_id	String	External id of operation must be unique for Partner

POST	
	<pre>{ "jsonrpc": "2.0", "method": "hold.dismiss", "id": {{randomInt}}, "params": { "ext_id": "{{unique_ext_id}}" } }</pre>

Sample responses

 Response hold.confirm method same as hold.get.otp response only differs with state and payment id, RRN

5. Payment

The method for payment with a commission.

Payment with OTP Implementation Scenario

Method for processing payments with OTP verification. This flow involves getting an OTP code, verifying it, and completing or canceling the payment.

Note: To perform the transaction, the E-POS must be registered in SV-Gate, HUMO, and VISA.

1. Request OTP:
 - 1.1. Call `payment.create` to initiate the payment and send an SMS with the OTP code to the client.
2. Verify Payment:
 - 2.1. Use `payment.verify` with the `ext_id` and the OTP code received by the client to confirm the transaction.
3. Cancel Payment (if needed):
 - 3.1. Call `payment.cancel` with the `ext_id` to cancel the transaction if it is no longer required (Humo payment cancellations are only allowed within the same month the transaction occurred. Transactions from a previous month can only be canceled through an official request submitted to HUMO.)

⚠ In case of a timeout or non-final states (e.g., 29, 30 - IN PROGRESS), use `payment.state` with the `ext_id` to poll for the final status (success or error).

5.1 Payment using form

This method creates a payment form that processes the operation using **3DS**. OTP handling is integrated into the form flow.

Payment Using Form Implementation Scenario

OTP verification is seamlessly integrated into the payment form workflow, designed to simplify the process and avoid PCI compliance requirements for partners who do not support 3DS directly.

1. Initiate Payment:
 - 1.1. Send a request to `payment.initiate` to generate a payment link.
 - 1.2. The link is provided in the response (link field).
2. Client Interaction:
 - 2.1. Share the link with the client.
 - 2.2. The client opens the form and enters their card details in the input fields provided.
 - 2.3. After submitting the card details, the client is redirected to the 3DS verification page (`payment.verify` form version).
3. Callback and Redirect:
 - 3.1. Upon success or failure, we send a callback to the URL provided in the `callback_url` parameter of `payment.initiate`.
 - 3.2. Additionally, we redirect the client to the `redirect_url` specified in the request.
4. Handling Non-Final States:
 - 4.1. If the callback contains non-final states (e.g., 29, 30 - IN PROGRESS), partners must use the `payment.state` method with the `ext_id` to poll for the final transaction status.

⚠ In cases of timeouts or incomplete states, partners should poll the `payment.state` method to determine whether the transaction was successful or failed.

This approach ensures a secure and efficient payment process while reducing the compliance burden for partners.

Request

jsonrpc	String	"2.0"
method	String	payment.initiate
id	String Integer	Request id
params	Object	
ext_id	String	External id of operation must be unique for Partner
receiver_id	String	Receiver id such as card number, account, etc. Note: The unique client code (receiver_id) —the end recipient of the funds, which allows identification within the framework of inter-system interaction. For example, 4. For mobile number payment—the full phone number; 5. For payment to an e-wallet—the full e-wallet number 6. For currency exchange operations: PINFL or the user's phone number, etc. This parameter is not processed or analyzed but is required to meet legal requirements for combating money laundering (AML).
amount	Integer	Amount must be evaluated in tiyins (UZS * 100) Example: For 10,000 UZS, the amount should be 1000000
currency	String	The currency code in ISO 4217 format (e.g., 860 for UZS, 840 for USD).
callback_url	String	The URL where the system will send the transaction result (success or failure). Used for asynchronous notification to the partner.
redirect_url	String	The URL to which the client will be redirected after completing the payment process. Useful for displaying a success or failure message to the user on the partner's interface.
phone_number	String	Sender's phone number should match regex '\+\d{12}\$'. Example: "+998999999999". Optional parameter. If you send this parameter the system will check whether the given phone number matches phone number of the card. In case of mismatch it will return different errors according to PC of the card (Uzcard or Humo)

POST

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "payment.initiate",
  "params": {
    "ext_id": "auto_generated_in_pre_request_script",
    "receiver_id": "receiver_phone",
    "amount": 100,
    "currency": "860",
    "callback_url": "my_web_service/webhook",
    "redirect_url": "my_web_service/status_page"
  }
}
```

Sample responses

In the response, please note that in the **initiated** state, certain data is not returned if the user has not proceeded with the payment form.

- If the user enters their card data and submits the form, we will return the relevant details (e.g., card mask, owner name, E-POS, etc.).
- However, if you check the state of a payment in the **initiated** status and the user has not entered or submitted their data, these fields will remain empty in the response.

ext_id	String	External id of operation must be unique for Partner
state	Integer	State of payment
number	String	Card number
description	String	Description of payment
amount	Integer	Amount
currency	String	Currency code, 860=UZS
commission	Integer	Commission
account	Object	
card	Object	
phone	Object	
owner	Object	
mask	String	
payment	Object	

ref_num	String	
id	Integer	
merchant	Object	
epos	Object	
type	Object	

```
{
  "jsonrpc": "2.0",
  "result": {
    "ext_id": "cc79d99-410d-a8db-61643fc8bbe4",
    "state": -1,
    "number": null,
    "description": "Initiated",
    "amount": 100,
    "currency": "860",
    "commission": 0,
    "link": "https://gw.cloudgate.uz/payment/verification?id=xxxx",
    "account": [
      {
        "name": "card",
        "title": {
          "ru": "Карта получателя",
          "en": "Receiver card",
          "uz": "Qabul qiluvchi kartasi"
        },
        "mask": "",
        "number": ""
      },
      {
        "name": "owner",
        "title": {
          "ru": "Получатель",
          "en": "Receiver",
          "uz": "Qabul qiluvchi"
        },
        "value": ""
      },
      {
        "name": "phone",
        "title": {
          "ru": "Номер телефона",
          "en": "Phone number",
          "uz": "Telefon raqami"
        },
        "mask": ""
      }
    ],
    "payment": {
      "ref_num": "UO-G15238521bc-7b32-475f",
      "id": null
    },
    "merchant": {
      "organization": "Unisoft",
      "epos": {
        "merchant": "",
        "terminal": ""
      },
      "type": {
```

```

        "ru": "Оплата",
        "en": "Payment",
        "uz": "To'lov"
    },
    },
    "id": "136",
    "status": true,
    "origin": "payment.create",
    "host": {
        "host": "unipos.online",
        "timestamp": "2023-05-15 05:59:10.969002"
    }
}

```

5.2 Create

Method creates payment cheque

Request

jsonrpc	String	"2.0"
method	String	payment.create
id	String Integer	Request id
params	Object	
ext_id	String	External id of operation must be unique for Partner
number	String	Plain card number 16 characters.
expire	String	Plain card expiry 4 characters in format "YYmm" e.g.: 2701.
receiver_id	String	Receiver id such as card number, account, etc. Note: The unique client code (receiver_id) —the end recipient of the funds, which allows identification within the framework of inter-system interaction. For example, 7. For mobile number payment—the full phone number; 8. For payment to an e-wallet—the full e-wallet number 9. For currency exchange operations: PINFL or the user's phone number, etc. This parameter is not processed or analyzed but is required to meet legal requirements for combating money laundering (AML).
receiver country	String	(UZS, US, RUS) abbreviation

amount	Integer	Amount must be evaluated in tiyins (UZS * 100)
merchant_id	String	Merchant ID
terminal_id	String	Terminal IID

POST	
	<pre>{ "jsonrpc": "2.0", "id": "{{randomInt}}", "method": "payment.get.otp", "params": { "ext_id": "{{randomUUID}}", "number": "{{number}}", "expire": "{{expire}}", "receiver_id": "{{expire}}", "receiver_country": "{{optional}}", "amount": 100, "merchant_id": "{{h_merchant}}", "terminal_id": "{{h_terminal}}" } }</pre>

Sample responses

ext_id	String	External id of operation must be unique for Partner
state	Integer	State of payment
number	String	Card number
description	String	Description of payment
amount	Integer	Amount
currency	String	Currency code, 860=UZS
commission	Integer	Commission
account	Object	
card	Object	
phone	Object	
owner	Object	
mask	String	
payment	Object	

ref_num	String	
id	Integer	
merchant	Object	
epos	Object	
type	Object	

```
{
  "jsonrpc": "2.0",
  "result": {
    "ext_id": "cc79d99-410d-a8db-61643fc8bbe4",
    "state": 0,
    "number": "8600482952**4119",
    "description": "created check",
    "amount": 100,
    "currency": "860",
    "commission": 0,
    "account": [
      {
        "name": "card",
        "title": {
          "ru": "Карта получателя",
          "en": "Receiver card",
          "uz": "Qabul qiluvchi kartasi"
        },
        "mask": "860048*****4119",
        "number": "860048**52964119"
      },
      {
        "name": "owner",
        "title": {
          "ru": "Получатель",
          "en": "Receiver",
          "uz": "Qabul qiluvchi"
        },
        "value": ""
      },
      {
        "name": "phone",
        "title": {
          "ru": "Номер телефона",
          "en": "Phone number",
          "uz": "Telefon raqami"
        },
        "mask": "*****9900"
      }
    ],
    "payment": {
      "ref_num": "U0-G15238521bc-7b32-475f",
      "id": null
    },
    "merchant": {
      "organization": "Unisoft",
      "epos": {
        "merchant": "904811434",
        "terminal": "92448569"
      }
    },
  },
}
```

```

        "type": {
            "ru": "Оплата",
            "en": "Payment",
            "uz": "To'lov"
        },
    },
    "id": "136",
    "status": true,
    "origin": "payment.create",
    "host": {
        "host": "unipos.online",
        "timestamp": "2023-05-15 05:59:10.969002"
    }
}

```

5.2.1 Create without 3DS

Method creates payment with card token

Request

jsonrpc	String	"2.0"
method	String	payment.create.wods
id	String Integer	Request id
params	Object	
ext_id	String	External id of operation must be unique for Partner
token	String	Plain card token 32 characters.
receiver_id	String	Receiver id such as card number, account, etc. Note: The unique client code (receiver_id) —the end recipient of the funds, which allows identification within the framework of inter-system interaction. For example, 10. For mobile number payment—the full phone number; 11. For payment to an e-wallet—the full e-wallet number 12. For currency exchange operations: PINFL or the user's phone number, etc. This parameter is not processed or analyzed but is required to meet legal requirements for combating money laundering (AML).
receiver country	String	(UZS, US, RUS) abbreviation
amount	Integer	Amount must be evaluated in tiyins (UZS * 100)
merchant_id	String	Merchant ID

terminal_id	String	Terminal ID
currency	String	The currency field specifies the currency in which the payment amount is denominated. It must be provided as an ISO 4217 numeric currency code. If not explicitly set, the default value '860' is used, which corresponds to Uzbekistan Som (UZS).

POST	
<pre>{ "jsonrpc": "2.0", "id": "{{randomInt}}", "method": "payment.create.wods", "params": { "ext_id": "{{randomUUID}}", "token": "{{token}}", "receiver_id": "{{expire}}", "receiver_country": "{{optional}}", "amount": 100, "merchant_id": "{{h_merchant}}", "terminal_id": "{{h_terminal}}", "currency": "{{currency}}", } }</pre>	

Sample responses

ext_id	String	External id of operation must be unique for Partner
state	Integer	State of payment
number	String	Card number
description	String	Description of payment
amount	Integer	Amount
currency	String	Currency code, 860=UZS
commission	Integer	Commission
account	Object	
card	Object	
phone	Object	

owner	Object	
mask	String	
payment	Object	
ref_num	String	
id	Integer	
merchant	Object	
epos	Object	
type	Object	

```
{
  "jsonrpc": "2.0",
  "result": {
    "ext_id": "cc79d99-410d-a8db-61643fc8bbe4",
    "state": 0,
    "description": "Created",
    "amount": 100,
    "currency": "860",
    "commission": 0,
    "account": [
      {
        "name": "card",
        "title": {
          "ru": "Карта получателя",
          "en": "Receiver card",
          "uz": "Qabul qiluvchi kartasi"
        },
        "mask": ""
      },
      {
        "name": "owner",
        "title": {
          "ru": "Получатель",
          "en": "Receiver",
          "uz": "Qabul qiluvchi"
        },
        "value": "{{card_owner}}"
      },
      {
        "name": "phone",
        "title": {
          "ru": "Номер телефона",
          "en": "Phone number",
          "uz": "Telefon raqami"
        },
        "mask": null
      }
    ],
    "payment": {
      "ref_num": "U0-G15238521bc-7b32-475f",
      "id": null
    },
    "merchant": {
```

```

    "organization": "Unisoft",
    "epos": {
      "merchant": "904811434",
      "terminal": "92448569"
    },
    "type": {
      "ru": "Оплата",
      "en": "Payment",
      "uz": "To'lov"
    }
  },
  "id": "136",
  "status": true,
  "origin": "payment.create.wods",
  "host": {
    "host": "unipos.online",
    "timestamp": "2025-05-15 05:59:10.969002"
  }
}

```

5.3 Confirm

This method confirms the payment, and the card will be charged instantly.

Request

jsonrpc	String	"2.0"
method	String	payment.verify
id	String Integer	Request id
params	Object	
ext_id	String	External id of operation must be unique for Partner

POST

```

{
  "jsonrpc": "2.0",
  "id": "{{randomInt}}",
  "method": "payment.verify",
  "params": {
    "ext_id": "{{unique_ext_id}}",
    "code": "{{OTP_CODE_CLIENT}}",
  }
}

```

Sample responses

Response cheque model same as create response

```
{
  "jsonrpc": "2.0",
  "result": {
    "ext_id": "cc74c-410d-a8db-61643fc8bbe4",
    "state": 4,
    "number": "86004829**964119",
    "description": "Successful transaction",
    "amount": 100,
    "currency": "860",
    "commission": 0,
    "account": [
      {
        "name": "card",
        "title": {
          "ru": "Карта получателя",
          "en": "Receiver card",
          "uz": "Qabul qiluvchi kartasi"
        },
        "mask": "860048*****4119",
        "number": "86004**952964119"
      },
      {
        "name": "owner",
        "title": {
          "ru": "Получатель",
          "en": "Receiver",
          "uz": "Qabul qiluvchi"
        },
        "value": ""
      }
    ],
    "payment": {
      "ref_num": "U5910-718521bc-7b32-475f",
      "id": "018300551155"
    },
    "merchant": {
      "organization": "Unisoft",
      "epos": {
        "merchant": "904811434",
        "terminal": "92448569"
      },
      "type": {
        "ru": "Оплата",
        "en": "Payment",
        "uz": "To'lov"
      }
    }
  },
  "id": "103",
  "status": true,
  "origin": "payment.verify",
  "host": {
    "host": "unipos.online",
    "timestamp": "2023-05-15 06:04:01.424125"
  }
}
```

5.3.1 Confirm Wods

This method confirms the payment, and the card will be charged instantly.

Request

jsonrpc	String	"2.0"
method	String	payment.confirm.wods
id	String Integer	Request id
params	Object	
ext_id	String	External id of operation must be unique for Partner

POST

```
{
  "jsonrpc": "2.0",
  "id": "{{${randomInt}}}",
  "method": "payment.confirm.wods",
  "params": {
    "ext_id": "{{unique_ext_id}}"
  }
}
```

Sample responses

Response cheque model same as create response

```
{
  "jsonrpc": "2.0",
  "result": {
    "ext_id": "cc74c-410d-a8db-61643fc8bbe4",
    "state": 4,
    "number": "86004829**964119",
    "description": "Successful transaction",
    "amount": 100,
    "currency": "860",
    "commission": 0,
    "account": [
      {
        "name": "card",
        "title": {
          "ru": "Карта получателя",
          "en": "Receiver card",
          "uz": "Qabul qiluvchi kartasi"
        },
        "mask": ""
      },
      {
        "name": "owner",
        "title": {
          "ru": "Получатель",

```

```

        "en": "Receiver",
        "uz": "Qabul qiluvchi"
    },
    "value": ""
  },
  ],
  "payment": {
    "ref_num": "U5910-718521bc-7b32-475f",
    "id": "018300551155"
  },
  "merchant": {
    "organization": "Unisoft",
    "epos": {
      "merchant": "904811434",
      "terminal": "92448569"
    },
    "type": {
      "ru": "Оплата",
      "en": "Payment",
      "uz": "To'lov"
    }
  },
  "id": "103",
  "status": true,
  "origin": "payment.confirm.wods",
  "host": {
    "host": "unipos.online",
    "timestamp": "2023-05-15 06:04:01.424125"
  }
}

```

5.4 Cancel

Method cancel payment and return payment amount totally to the card owner

Request

jsonrpc	String	"2.0"
method	String	payment.cancel
id	String Integer	Request id
params	Object	
ext_id	String	External id of operation must be unique for Partner

POST

```

{
  "jsonrpc": "2.0",

```

POST

```
{
  "id": "{{randomInt}}",
  "method": "payment.cancel",
  "params": {
    "ext_id": "{{ext_id}}"
  }
}
```

Sample responses

Response cheque model same as payment.create response

```
{
  "jsonrpc": "2.0",
  "result": {
    "ext_id": "cc79d992-114c-410d-a8db-61643fc8bbe4",
    "state": 21,
    "number": "8600482952964119",
    "description": "payment canceled",
    "amount": 100,
    "currency": "860",
    "commission": 0,
    "account": [
      {
        "name": "card",
        "title": {
          "ru": "Карта получателя",
          "en": "Receiver card",
          "uz": "Qabul qiluvchi kartasi"
        },
        "mask": "860048*****4119",
        "number": "8600482952964119"
      },
      {
        "name": "owner",
        "title": {
          "ru": "Получатель",
          "en": "Receiver",
          "uz": "Qabul qiluvchi"
        },
        "value": ""
      }
    ],
    "payment": {
      "ref_num": "UO-GR-H-051523055910-718521bc-7b32-475f",
      "id": "018300551155"
    },
    "merchant": {
      "organization": "Unisoft",
      "epos": {
        "merchant": "904811434",
        "terminal": "92448569"
      },
      "type": {
        "ru": "Оплата",
        "en": "Payment",
        "uz": "To'lov"
      }
    }
  },
}
```

```
{
  "id": "665",
  "status": true,
  "origin": "payment.cancel",
  "host": {
    "host": "unipos.online",
    "timestamp": "2023-05-15 06:05:13.957347"
  }
}
```

5.4 State

Method returns payment cheque state directly from Processing Centre

Request

jsonrpc	String	"2.0"
method	String	payment.state
id	String Integer	Request id
params	Object	
ext_id	String	External id of operation must be unique for Partner

POST	
<pre><code>{ "jsonrpc": "2.0", "id": "{{randomInt}}", "method": "payment.state", "params": { "ext_id": "{{ext_id}}" } }</code></pre>	

Sample responses

Response cheque model same as [payment.create](#) response

6. Transfer

The method for card-to-card transfers. This method enables money transfer operations from one card to another using the sender's token and the recipient's token or card number.

Note: To perform the transaction, the E-POS must be registered in SV-Gate, HUMO and VISA.

Transfer (P2P) Implementation Scenario:

1. Get Access Token: Authenticate and authorize API requests.

2. Register Sender Card: Ensure the sender's card is registered and associated with a token.
3. Create Transfer:
 - 3.1. Use the sender's token to call `transfer.p2p.create`.
 - 3.2. Optionally, use `card.info` to verify the recipient's card if needed.
 - 3.3. The transfer creation validates card details, partner/bank limits, and compliance checks.
 - 3.4. Note: State 0 (CREATED) indicates successful creation but does not transfer funds yet.
4. Confirm Transfer:
 - 4.1. If the transfer is in state 0, call `transfer.p2p.confirm` to complete the transaction.

⚠ In case of a timeout or non-final states (e.g., 29, 30 - IN PROGRESS), use [transfer.state](#) to poll for the final status (success or error).

6.1 Create

Method creates cheque for P2P transfer

Request

jsonrpc	String	"2.0"
method	String	<code>transfer.create</code>
id	String Integer	Request id
params	Object	
ext_id	String	External id of operation must be unique for Partner
number	String	Plain card number 16 characters.
expire	String	Plain cards expiry 4 characters in format "YYmm" eg: 2701.
amount	Integer	Amount must be evaluated in tiyins (UZS * 100)
merchant_id	String	Merchant ID
terminal_id	String	Terminal IID

POST

```
{
  "jsonrpc": "2.0",
  "id": "{{randomUUID}}",
  "method": "transfer.create",
  "params": {
    "ext_id": "{{randomUUID}}",
    "number": "{{number}}",
    "receiver": "{{receiver_card_number}}",
    "expire": "{{expire}}",
    "amount": 100,
  }
}
```


POST	
	<pre> "merchant_id": "{{merchant_id}}", "terminal_id": "{{terminal_id}}" } } </pre>

Sample responses

Response cheque model same with which methods create response

6.2 Confirm

Method confirms cheque and proceeds transfer

Request

jsonrpc	String	"2.0"
method	String	transfer.confirm
id	String Integer	Request id
params	Object	
ext_id	String	External id of operation must be unique for Partner

POST	
	<pre> { "jsonrpc": "2.0", "id": "{{randomUUID}}", "method": "transfer.confirm", "params": { "ext_id": "{{ext_id}}" } } </pre>

Sample responses

Response cheque model same with which methods create response

6.3 Cancel

Method cancels operation

Request

jsonrpc	String	"2.0"
---------	--------	-------

method	String	<code>transfer.cancel</code>
id	String Integer	Request id
params	Object	
ext_id	String	External id of operation must be unique for Partner

POST	
<pre>{ "jsonrpc": "2.0", "id": "{{randomUUID}}", "method": "transfer.cancel", "params": { "ext_id": "{{ext_id}}" } }</pre>	

Sample responses

Response cheque model same with which methods create response

6.4 State

Method returns the state of transfer

Request

jsonrpc	String	<code>"2.0"</code>
method	String	<code>transfer.state</code>
id	String Integer	Request id
params	Object	
number	String	Plain card number 16 characters.
expire	String	Plain cards expiry 4 characters in format "YYmm" e.g.: 2701.

POST	
<pre>{ "jsonrpc": "2.0", "id": "{{randomUUID}}", "method": "transfer.state", "params": { "ext_id": "{{ext_id}}" } }</pre>	

POST	
{	

Sample responses

Response cheque model same as create response

7. Account to Card

Method for replenishing the card balance. This method enables balance replenishment operations using the recipient's token or card number.

Note: To perform a transaction, the E-POS must be registered with SV-Gate, HUMO, VISA Korti Milli, Mbank and Elkart.

Account to Card (A2C) Implementation Scenario:

1. Get Access Token: Authenticate and authorize API requests.
2. Validate Receiver Card (Optional): Use card.info to verify card validity or cardholder details (FIO).
3. Create Cheque:
 - 3.1. Call transfer.credit.create to initiate a cheque.
 - 3.2. The cheque creation validates the card, checks partner/bank limits, ensures AML compliance, and processes conversion rules.
 - 3.3. Note: State 0 (CREATED) indicates a successful cheque creation but does not transfer funds immediately.
4. Confirm Transfer:
 - 4.1. If the cheque is in state 0, call transfer.credit.confirm to transfer funds to the card.

 If transfer.credit.confirm results in a timeout or non-final states (e.g., 29, 30 - IN PROGRESS), use transfer.credit.state to poll for a final status (success or error).

 The single card limit for A2C transfers using a registered EPOS is **145 million UZS** for monthly transactions.

7.1 Create

This method creates an Account-to-Card (A2C) cheque to transfer funds directly to a recipient's card. It supports Uzcard, Humo, and Visa Direct transactions, with specific requirements based on the card type.

1. **Merchant & Terminal IDs:**
 - For partners with multiple merchants or EPOS terminals, you must include **merchant_id** and **terminal_id** in the request.
 - If a single merchant is used, these fields may be auto-settled from the **Unisoft** side.
2. **Receiver Country:**
 - **receiver_country** (*value is configured to FRAUD by default, so we set as receiver instead of sender*) is optional for Uzcard, Humo and Visa transactions.
 - Including **receiver_country** improves AML (Anti-Money Laundering) compliance and enhances transaction reporting to the **Central Bank (CBU)** and **Universalbank's AML** department.
3. **Visa Top-Ups:**

To comply with Visa regulations, you must include the following additional fields in the request:

 - **Sender Block:**
 - passport details (document type, series number, expiry date).

- first_name, last_name of the sender.
- ip address and sender_id (unique user ID from the partner's system).
- Receiver Block:
 - first_name and last_name of the receiver are mandatory for Visa transactions.
(Sender must fill these fields)

POST

Sample for VISA top up

```
{
  "jsonrpc": "2.0",
  "id": "123",
  "method": "transfer.credit.create",
  "params": {
    "ext_id": "unique_operation_id",
    "number": "{{visa_card_number}}",
    "amount": 1200,
    "currency": "840",
    "pc_typ": 5,
    "receiver_country": "RUS",
    "sender": {
      "pinfl": "21321321321321",
      "passport_series": "AB5889688",
      "birth_date": "30.08.1999",
      "first_name": "John",
      "last_name": "Doe",
      "sender_id": 1,
      "phone": "+998935989900",
      "ip": "8.8.8.8"
    },
    "receiver": {
      "first_name": "Ivan",
      "last_name": "Ivanov"
    }
  }
}
```

4. Uzcard & Humo Top-Ups:

- The sender's detailed personal information and the receiver block are optional.
- For faster and simpler processing, fewer fields are required compared to Visa.

POST

Sample for Uzcard and Humo top up

```
{
  "jsonrpc": "2.0",
  "id": "123",
  "method": "transfer.credit.create",
  "params": {
    "ext_id": "unique_operation_id",
    "number": "{{uzcard_or_humo_card_number}}",
    "amount": 10000,
    "currency": "860",
    "receiver_country": "RUS",
  }
```

POST	Sample for Uzcard and Humo top up
	<pre> "sender": { //or send as "passport" "nationality": "Uzbek", "type": "passport", "seriesNumber": "AB1234567", "birthDate": "30.08.1999", "validTo": "27.06.2026", "pnfl": "3300*****890026" } } </pre>

POST	Sample for Using phone number Uzcard/Humo top up
	<pre> { "jsonrpc": "2.0", "id": "123", "method": "transfer.credit.create", "params": { "ext_id": "unique_operation_id", "number": "{{uzb_phone_number}}", "bank_code": "20025", "amount": 10000, "currency": "860", "receiver_country": "RUS", "sender": { //or send as "passport" "nationality": "Uzbek", "type": "passport", "seriesNumber": "AB1234567", "birthDate": "30.08.1999", "validTo": "27.06.2026", "pnfl": "3300*****890026" } } } </pre>

POST	Sample for Using phone number VISA top up
	<pre> { "jsonrpc": "2.0", "id": "123", "method": "transfer.credit.create", "params": { "ext_id": "unique_operation_id", "number": "{{uzb_phone_number}}", "pc_type": 5, "amount": 10000, "currency": "860", "receiver_country": "RUS", "sender": { //or send as "passport" "nationality": "Uzbek", "type": "passport", "seriesNumber": "AB1234567", "birthDate": "30.08.1999", </pre>

POST	Sample for Using phone number VISA top up
	<pre> "validTo": "27.06.2026", "pnfl": "3300*****890026" } } } </pre>

POST	Sample for Using phone or card Korti Milli
	<pre> { "jsonrpc": "2.0", "id": "123", "method": "transfer.credit.create", "params": { "ext_id": "unique_operation_id", "number": "{{tj_card_number}}", "pc_type": 7, "amount": 10000, "currency": "860", "receiver_country": "TJS", "sender": { //or send as "passport" "nationality": "Uzbek", "type": "passport", "seriesNumber": "AB1234567", "birthDate": "30.08.1999", "validTo": "27.06.2026", "pnfl": "3300*****890026", "phone": "9989900000", "first_name": "John", "last_name": "Doe", } } } </pre>

POST	Sample for Using phone or card Mbank and Elkart
	<pre> { "jsonrpc": "2.0", "id": "123", "method": "transfer.credit.create", "params": { "ext_id": "unique_operation_id", "number": "{{kgz_phone_number_or_card_number}}", "pc_type": 5, "amount": 10000, "currency": "860", "receiver_country": "RUS", "sender": { //or send as "passport" "nationality": "Uzbek", } } } </pre>

POST
Sample for Using phone or card Mbank and Elkart

```

    "type": "passport",
    "seriesNumber": "AB1234567",
    "birthDate": "30.08.1999",
    "validTo": "27.06.2026",
    "pnfl": "3300*****890026",
    "phone": "9989900000",
    "first_name": "John",
    "last_name": "Doe",
  }
}

```

Request

jsonrpc	String	M	"2.0"
method	String	M	transfer.credit.create
id	String Integer	M	Request id
params	Object		
ext_id	String	M	External id of operation must be unique for Partner
number	String	M	Plain card number 16 characters or Uzbekistan Phone number
bank_code	String	C	Required when a phone number is provided in the number field. Specifies the bank (Bank codes) for the top-up (e.g., "20025" selects a specific bank). If not provided, the top-up will proceed in a random order.
amount	String	M	Amount must be evaluated in tiyins (UZS * 100)
currency	String	O	Currency code in ISO 4217 format (e.g., "840" for USD, "860" for UZS, "643" for RUBLE).
receiver_country	String	O	ISO country code of the sender - <u>value is configured to FRAUD by default, so we set as receiver</u> (e.g., "RUS" for Russia).
merchant_id	String	C	Merchant ID
terminal_id	String	C	Terminal IID
passport or sender	Object	C	Information about the sender (payer).
nationality	String	C	Nationality of the sender (e.g., "Uzbek").

type	String	C	Document type ("passport" or "ID").
seriesNumber	String	C	Series and number of the sender's document (e.g., passport number).
birthDate	String	C	Sender's date of birth (format: DD.MM.YYYY). (required for Visa transactions).
validTo	String	C	Document expiration date (format: DD.MM.YYYY).
pinfl	String	C	Sender's personal identification number (PINFL).
first_name	String	C	Sender's first name (required for Visa transactions).
last_name	String	C	Sender's last name (required for Visa transactions).
sender_id	String	C	Unique user ID from the partner's system for the sender. (mandatory for Visa card top-ups).
phone	String	C	Sender's phone number (e.g., +998935989900). (required for Visa transactions).
ip	String	C	Sender's IP address (e.g., "8.8.8.8"). (required for Visa transactions).
receiver	Object	C	Information about the receiver (required for Visa transactions).
first_name	String	C	Receiver's first name (required for Visa transactions).
last_name	String	C	Receiver's last name (required for Visa transactions).
pc_type	String	C	Required to pass value - 5 to top up VISA card by phone number

POST	
	<pre> { "jsonrpc": "2.0", "id": "123", "method": "transfer.credit.create", "params": { "ext_id": "auto_generated_in_pre_request_script", "number": "{{card_number_visa}}", "amount": 12, "currency": "840", "receiver_country": "RUS", "sender": { "nationality": "Uzbek", "type": "passport", "seriesNumber": "AB1234567", "birthDate": "30.08.1999", "validTo": "27.06.2026", "pnfl": "3300*****890026", "first_name": "{{randomFirstName}}", "last_name": "{{randomLastName}}", "sender_id": 1, "phone": "+998935989900", "ip": "8.8.8.8" } } } </pre>

POST

```
{
  "receiver": {
    "first_name": "{{ $randomFirstName }}",
    "last_name": "{{ $randomLastName }}"
  }
}
```

Sample responses

ext_id	String	External id of operation
state	Integer	State of operation
number	String	Receiver card number or Phone number
description	String	Operation state description
amount	Integer	Amount in UZS with precision tiyin
currency	Integer	Credit currency
commission	Float	Calculated commission
account	List	Receiver account information
payment	Object	
ref_num	String	Reference id of payment in PC (Uzcard, Humo, Visa, Tajikistan)
id	String	Payment id of operation - RRN
merchant	Object	
organization	String	
epos	Object	
merchant	String	
terminal	String	
type	Object	

```
{
  "jsonrpc": "2.0",
  "result": {
    "ext_id": "cf2a05b1-88a4-42d9-a794-500d6fd4ba55",
  }
}
```

```
"state": 0,
"number": "8600482952964119",
"description": "Cheque Created!",
"amount": 100,
"currency": "860",
"commission": 1.0,
"account": [
  {
    "name": "card",
    "title": {
      "ru": "Карта получателя",
      "en": "Receiver card",
      "uz": "Qabul qiluvchi kartasi"
    },
    "mask": "860048*****4119",
    "number": "8600482952964119"
  },
  {
    "name": "owner",
    "title": {
      "ru": "Получатель",
      "en": "Receiver",
      "uz": "Qabul qiluvchi"
    },
    "value": "ABDUL QODIR HUSANBOY"
  }
],
"payment": {
  "ref_num": "UO-TT-C-051523052340-7bfe0a72-6dd5-46e6",
  "id": null
},
"merchant": {
  "organization": "Unisoft",
  "epos": {
    "merchant": "90489288",
    "terminal": "92414051",
    "account": "29xxxxxxxx000111",
  },
  "type": {
    "ru": "Пополнения карта",
    "en": "Top-up card",
    "uz": "Karta xisobini to'ldirish"
  }
},
"id": "c4f15cbe-c6b3-40fa-a493-6423b6851531",
"status": true,
"origin": "transfer.credit.create",
"host": {
  "host": "unipos.online",
  "timestamp": "2023-05-15 05:23:40.219200"
}
}
```

7.2 Confirm

Method confirms cheque and funds client card

Request

jsonrpc	String	"2.0"
method	String	transfer.credit.confirm
id	String Integer	Request id
params	Object	
ext_id		External id of operation

POST

```
{
  "jsonrpc": "2.0",
  "id": "{{ $randomUUID }}",
  "method": "transfer.credit.confirm",
  "params": {
    "ext_id": "{{ ext_id }}"
  }
}
```

Response cheque model same as [transfer.credit.create](#) response

7.3 Cancel

HUMO cards:

- Partners must contact the support team and provide case details for cancellation.
- Our team will send an official letter to the HUMO processing center to finalize the cancellation.

Uzcard cards:

- Partners can directly call the cancellation method via API.

⚠ Note: Cancellation for **A2C** is not recommended as it is not 100% guaranteed. We advise avoiding this process whenever possible.

Request

jsonrpc	String	"2.0"
---------	--------	-------

method	String	transfer.credit.cancel
id	String Integer	Request id
params	Object	
ext_id	String	External id of operation

POST	
<pre>{ "jsonrpc": "2.0", "id": "{{randomUUID}}", "method": "transfer.credit.cancel", "params": { "ext_id": "{{ext_id}}" } }</pre>	

7.4 State

Method returns cheque state

Request

jsonrpc	String	"2.0"
method	String	transfer.credit.state
id	String Integer	Request id
params	Object	
ext_id	String	External id of operation must be unique for Partner

POST	
<pre>{ "jsonrpc": "2.0", "id": "{{randomUUID}}", "method": "transfer.credit.state", "params": { "ext_id": "{{ext_id}}" } }</pre>	

Response cheque model same as [transfer.credit.create](#) response

8. Informational methods

8.1 Rate

The `get.rate` method is used to fetch current exchange rates for financial transactions. The response provides detailed information about credit and debit rates, supporting calculations for multi-currency operations.

This method ensures accurate rate application for operations like payments, holds, and account-to-card (A2C) transfers.

⚠ Note! If you receive an empty result, please contact the administrator to configure your exchange rate settings.

Request

jsonrpc	String	"2.0"
method	String	<code>get.rate</code>
id	String Integer	Request id
params	Object	

POST

```
{
  "jsonrpc": "2.0",
  "id": "{{${randomUUID}}}",
  "method": "get.rate",
  "params": {}
}
```

Sample responses

transaction_type	String	Indicates the type of transaction (credit or debit).
is_primary	Boolean	Boolean indicating whether the currency is primary in the registry. Used to fix the rate for transactions where no actual exchange operation was performed. Each partner can have only one such primary rate (<code>"is_primary": true</code>) for every combination of transaction type and base currency.

currency	String	ISO 4217 code of the currency (840 for USD). Code of the base currency. Used for filtering the exchange rate returned by iABS (the internal system). Typically matches the currency code defined in iABS.
target_currency	String	ISO 4217 code of the currency (860 for UZS). Code of the target currency relative to which the base currency is expressed in iABS.
operator	String	The operation for rate conversion (e.g., multiply). defines how to convert from currency to target_currency. "multiply": $\text{target_amount} = \text{base_amount} * \text{value}$ "divide": $\text{target_amount} = \text{base_amount} / \text{value}$ The operator depends on how the rate is stored in iABS. The order of operands is fixed: base_amount always refers to currency, and the result is expressed in target_currency.
value	Integer	The exchange rate value. Used as the second operand in the operation defined by operator to convert from currency to target_currency .

Response

```
{
  "jsonrpc": "2.0",
  "result": [
    {
      "transaction_type": "credit",
      "is_primary": true,
      "currency": "840",
      "target_currency": "860",
      "operator": "multiply",
      "value": 12854.0
    },
    {
      "transaction_type": "debit",
      "is_primary": true,
      "currency": "840",
      "target_currency": "860",
      "operator": "multiply",
      "value": 13022.0
    }
  ],
  "id": "111",
  "status": true,
  "origin": "get.rate",
  "host": {
    "host": "Unipos",
    "timestamp": "2024-12-23 12:21:17.265255"
  }
}
```


8.2 Bin List

The bin.list method retrieves a list of BIN (Bank Identification Number) details, including the payment card type, issuing bank, and provider information. This is used to identify and classify cards for various operations.

⚠️ Note! Bins do not change frequently. You can fetch the data once per month or as needed. Any updates or new BIN values will be communicated accordingly.

If you send pc_type, the bins of the pc you sent will be returned. If you send params, all the bins we have will be returned.

Request

jsonrpc	String	"2.0"
method	String	bin.list
id	String Integer	Request id
params	Object	
pc_type	String	Pc type id
pc_type	Integer	1 - Uzcard 3 - Humo 5 - Visa 7 - Tajikistan 8 - Kyrgyzstan

POST	
<pre>{ "jsonrpc": "2.0", "id": "{{randomUUID}}", "method": "bin.list", "params": { "pc_type": 8 } }</pre>	

Sample responses

bin	String	The Bank Identification Number.
pc_type	Integer	The type of payment card: 1: Uzcard. 3: Humo. 5: VISA (soon)

		7. Tajikistan (somon) 7. Kyrgyzstan (som)
bank_name	String	The name of the issuing bank.
provider	String	The card provider (uzcard or humo)

Response

```
{
  "jsonrpc": "2.0",
  "result": [
    {
      "bin": "98602302",
      "pc_type": 3,
      "bank_name": "Universalbank",
      "provider": "humo"
    },
    {
      "bin": "98602466",
      "pc_type": 3,
      "bank_name": "Ravnaqbank",
      "provider": "humo"
    },
    {
      "bin": "6262830",
      "pc_type": 1,
      "bank_name": "Universal bank",
      "provider": "uzcard"
    },
    {
      "bin": "54408119",
      "pc_type": 1,
      "bank_name": "Ravnaq bank",
      "provider": "uzcard"
    }
  ],
  "id": "111",
  "status": true,
  "origin": "bin.list",
  "host": {
    "host": "Unipos",
    "timestamp": "2024-12-25 05:09:20.704214"
  }
}
```

8.3 Balance

Method returns account balance which is used as a deposit for A2C

8.4 Registry

1. File Format:

- Type: Excel (.xlsx)
- Delivery Method: Sent via email.
- Purpose: Registry of transactions between Universal Bank and its partner

2. File Naming Convention:

- universal_bank_<partner_name>_<start_date>_<end_date>.xlsx
- start_date: Start date of the registry period in the format YYYY_MM_DD.
- end_date: End date of the registry period in the format YYYY_MM_DD.
- Example: universal_bank_stripe_2024_12_20_2024_12_20.xlsx

Account to Card file columns description

Column	Description	Sample
payment_id	Unique identifier for the payment transaction.	123400000000
sender_fullname	Full name of the sender initiating the transaction.	ALI VALIYEV
payment_unique	Unique identifier assigned to the payment.	UO-TT-C-122024020026-85
partner_transaction_number	Transaction ext_id assigned by the partner system.	ext_id_12345
payment_date	Date of the payment transaction.	20-12-2024
payment_time	Time of the payment transaction.	07:00:26
payment_created_date	Date when the payment record was created.	20-12-2024
payment_created_time	Time when the payment record was created.	07:00:26
payment_system	Payment system used for the transaction (e.g., Uzcard, Humo).	Uzcard
operator_name	Name of the operator processing the payment.	Stripe
amount	Amount of the transaction in the local currency.	100228
transferred_currency	Currency in which the amount was transferred.	UZS

amount_in_usd	Equivalent amount in USD.	7.816283241
currency_rate	Conversion rate used for the transaction.	12823
total_amount	Total amount including any additional charges in the local currency.	100729.341
total_amount_in_usd	Total amount including charges in USD.	7.855364657
commission_amount	Commission charged for the transaction in the local currency.	501.141
commission_amount_in_usd	Commission charged for the transaction in USD.	0.039081416
request_currency	Currency requested for the transaction.	UZS
request_amount	Amount requested by the sender in the specified currency.	100228.2
description	Description or status of the transaction (e.g., Successful transaction).	Successful transaction
sender_card	Masked card number of the sender.	522123*****1234
sender_card_issuer	Issuer of the sender's card (e.g., Mastercard).	MASTERCARD
sender_country	Country of the sender initiating the transaction (e.g., RUS for Russia).	RUS

Payment registry file specs

- The main difference from A2C registry is the file name
universal_bank_<partner_name>**from_<start_date>_<end_date>**.xlsx

Column Name	Description	Sample
payment_id	Unique identifier for the payment transaction.	1234000000
sender_fullname	Full name of the sender initiating the transaction.	VALI ALIYEV
payment_unique	Unique identifier assigned to the payment.	UO-TT-H-122024020917-85c
partner_transaction_number	Transaction ext_id assigned by the partner system	ext_id_12345

payment_date	Date of the payment transaction.	20-12-2024
payment_time	Time of the payment transaction.	7:09:17
payment_created_date	Date when the payment record was created.	20-12-2024
payment_created_time	Time when the payment record was created.	7:09:17
payment_system	Payment system used for the transaction (e.g., Uzcard, Humo).	Uzcard
operator_name	Name of the operator processing the payment.	Stripe
amount	Amount of the transaction in the local currency.	170000.1
transferred_currency	Currency in which the amount was transferred.	UZS
amount_in_usd	Equivalent amount in USD.	13.08699769
currency_rate	Conversion rate used for the transaction.	12990
total_amount	Total amount including any additional charges in the local currency.	171275.1008
total_amount_in_usd	Total amount including charges in USD.	13.18515017
commission_amount	Commission charged for the transaction in the local currency.	1275.00075
commission_amount_in_usd	Commission charged for the transaction in USD.	0.098152483
request_currency	Currency requested for the transaction.	UZS
request_amount	Amount requested by the sender in the specified currency.	170000.1
description	Description or status of the transaction (e.g., Successful transaction).	Успешно
receiver_card	Masked card number of the receiver.	9016****123
receiver_card_issuer	Issuer of the receiver's card.	Not Specified
receiver_country	Country of the receiver initiating the transaction.	Not Specified

8.5 Registry

This method is used to retrieve the registry of transactions between **Universal Bank** and its partner via API.

It returns detailed transaction data (amounts, dates, statuses) in JSON format for reconciliation purposes.

Request

jsonrpc	String	"2.0"
method	String	get.registry.a2c
id	String Integer	Request id
params	Object	

POST	
<pre>{ "jsonrpc": "2.0", "id": "{{\${randomUUID}}}", "method": "get.registry.a2c", "params": { "datetime_from": "2025-01-05 12:00:00", "datetime_to": "2025-10-29 23:59:59" } }</pre>	

Sample responses

datetime_from	String	
datetime_to	String	

Response

<pre>{ "jsonrpc": "2.0", "result": [{ "ext_id": "rf_test_59", "ref_num": "U0-TT-C-052925072303-058e6a35-0f5d-43c4", "state": 4, "rrn": "024938144449", "amount": 200000000, "created_at": "2025-05-29T12:23:03.395327+05:00", "paid_at": "2025-05-29T12:23:15.087842+05:00" }], }</pre>	
---	--

```
{
  "ext_id": "rf_test_2",
  "ref_num": "UO-TT-C-060325104129-5641c580-9ec8-43b8",
  "state": 4,
  "rrn": " ",
  "amount": 1000,
  "created_at": "2025-06-03T15:41:29.611302+05:00",
  "paid_at": "2025-06-03T15:41:38.254083+05:00"
},
{
  "ext_id": "test_ext_id__2",
  "ref_num": "UO-TT-C-062425122151-355153bb-ebf8-4b6a",
  "state": 30,
  "rrn": null,
  "amount": 10000,
  "created_at": "2025-06-24T12:21:51.542388+05:00",
  "paid_at": "2025-06-24T12:22:36.192839+05:00"
}
],
"id": "1111",
"status": true,
"origin": "get.registry.a2c",
"host": {
  "host": "Unipos",
  "timestamp": "2025-10-13 17:46:08.303650"
}
}
```

9. Alias

9.1 Set

Request

jsonrpc	String	"2.0"
method	String	vads.aliasSet
id	String Integer	Request Id
params	Object	
card_token	String	Token of the card
phone	String	Phone Number

POST	
	<pre>{ "jsonrpc": "2.0", "id": 123, "method": "vads.aliasSet", "params": { "card_token": "", "phone": "998974559530" } }</pre>

9.2 Delete

Request

jsonrpc	String	"2.0"
method	String	vads.aliasDelete
id	String Integer	Request Id
params	Object	
phone	String	Phone Number
jsonrpc	String	"2.0"

POST	
	<pre>{ "jsonrpc": "2.0", "id": 123, "method": "vads.aliasDelete", "params": { "phone": "998974559530" } }</pre>

10. Cabinet

The Cabinet Beta Version is now live! This platform is designed to give you full visibility into your account transactions and payments. With the Cabinet, you can:

- Monitor Transactions: Track all account activities and payment operations.
- Access Detailed Information: View comprehensive details for each transaction, including statuses.
- Improve Financial Oversight: Gain insights that help you manage your financial operations more effectively.

10.1 How to access

For access and further instructions, please contact your coordinator. Enjoy the enhanced monitoring capabilities!

States

Card states

State	Message	Description
Uzcard card statuses		
0	VALID_CARD	
1	CALL_ISSUER	
2	WARM_CARD	
3	DO_NOT_HONOR	
4	HONOR_WITH_ID	
5	NOT_PERMITTED	
6	LOST_CARD_CAPTURE	
7	STOLEN_CARD_CAPTURE	
8	CALL_SECURITY_CAPTURE	
9	INVALID_CARD_CAPTURE	
10	PICK_UP_CARD_SPECIAL_CONDITION	
11	CALL_ACQUIRER_SECURITY	

12	TEMPORARY_BLOCKED_BY_USER	
13	PIN_ATTEMPTS_EXCEEDED	
14	FORCED_PIN_CHANGE	
15	CREDIT_DEBTS	
16	UNKNOWN	
17	PIN_ACTIVATION	
18	INSTANT_CARD_PERSONIFICATION_WAITING	
19	FRAUD_PREVENTION	
20	TEMPORARY_BLOCKED_BY_CLIENT	
21	PERMANENT_BLOCKED_BY_CLIENT	

Humo card statuses

000	Approved	
104	Decline, restricted card	
125	Decline, card not effective	
204	Pick-up, restricted card	
207	Pick-up, special conditions	
208	Pick-up, lost card	
209	Pick-up, stolen card	
280	Decline, Card is not active at Bank will	
281	Decline, Card is not active at Cardholder will	

Hold states

State	Final	Description
-1		Received to Process
-2		Holding permission pending
0		Created
3		Hold successfully

4	+	Confirmed
5	+	Dismissed
6	+	Dismissed
21	+	Cancelled
30		In Progress

Payment states

State	Final	Description
-1		Received to Process
-98	+	Not created
0		Created
1		Processing
4	+	Success
5	+	Returned
21	+	Cancelled
22	+	Cheque expired
29		In progress
30		In progress
33	+	transaction error
34	+	Wrong OTP Code

Account To card states

State	Final	Description
-1		Received to Process
-98	+	Not created
0		Created
1		Sent to PC (in progress)
4	+	Success
21	+	Cancel

22	+	Cheque expired
29		In progress
30		In progress
33	+	transaction error

Bank Codes

Bank code	Bank Name
20001	Ipak yoli
20002	Agrobank
20003	Turonbank
20004	Ipotekabank
20005	Asaka bank
20006	Infin bank
20007	NBU
20008	Hamkorbank
20009	Xalq bank
20010	UzPSB
20011	Trustbank
20012	Aloqa bank
20013	Mikrokredit bank
20014	Asia Alliance bank
20015	Orient finance
20016	Poytaxt bank
20017	Tenge bank
20018	Davr bank
20019	Universal bank
20020	TBCBank
20021	Kapital bank
20022	Ravnaqbank
20023	Uzagroexportbank
20024	Ziraatbank
20025	Universalbank
20026	Anor bank
20027	Turkistonbank

20028	Madad invest bank
20029	Savdogarbank
20030	KDB Bank Uzbekistan
20031	Saderatbank
20032	Hi-Tech bank
20033	AO UZUM BANK
20034	Smartbank

Error Codes

Code	Description	Scenario	Partner Action
Basic error codes and their descriptions			
-500	Service is temporarily unavailable!	The service is down due to maintenance or a temporary issue.	Retry later. Implement retry logic with exponential backoff. If the issue persists, contact support.
-32700	Request body incorrect	Malformed JSON or missing required parameters in the request.	Fix request format. Validate and correct the request body format, then resend the request.
-32101	Username or password incorrect	Incorrect credentials provided during authentication.	Check credentials. Verify username/password and try again. Contact support if issues persist.
-32102	Use token in your headers with given for you 'token' to authorize	Authorization token is missing in the request headers.	Add token. Include the provided token in the request headers and resend the request.
-32103 -36042	Authorization token expired or not valid	The provided token has expired or is invalid.	Refresh token. Request a new token and retry the request with the updated token.
-32301	Merchant or terminal exists!	Attempt to create a merchant or terminal that already exists.	Avoid duplicate creation. Check existing merchants/terminals before creating new ones.
-32302	Merchant or terminal not exists!	Attempt to update or reference a	Verify existence. Confirm the

		merchant or terminal that doesn't exist.	merchant/terminal exists before performing operations.
-32303	Merchant and terminal inactive!	The merchant or terminal is inactive and cannot process requests.	Reactivate. Ensure the merchant/terminal is active before retrying the request.
-32401	External ID already exists!	Attempt to use an External ID that has already been registered.	Use unique External ID. Generate a new unique External ID and resend the request.
-32402	External ID not found!	Attempt to reference an External ID that doesn't exist in the system.	Check ID validity. Verify the External ID exists before attempting the operation.
-32410	The phone number attached to the card and the requesting phone number do not match!	Inform user to change the card or registration phone number	
-32510	Receiver not found	The receiver account/card does not exist.	Verify the receiver details (card number/account/phone/ID).

Svgate error codes and their descriptions

05	Do not honor transaction		
08	Approve with identification		
12	Transaction needs to be entered again		
12	Invalid transaction - retry		
13	Cannot process amount. This code is only for truly erroneous amounts, i.e, exceeds machine capabilities or \$0 cash withdrawal. No limits involved.		
14	Invalid account - retry		
14	Forced post: No card on file		
15	The card is already active		
30	The message received was not within standards		
31	Issuer inoperative		

31	SmartVista not permitted to stand in		
33	Card expired, capture card		
36	Account restricted, capture card		
37	Call Security - Capture		
41	Lost Card - Capture		
43	Stolen Card - Capture		
51	Insufficient funds - retry		
55	Incorrect PIN, foreign.		
57	Not permitted		
57	Transaction not permitted by law		
57	Account restricted		
61	Negative auth usage cycle limit exceeded		
61	Card's ATM or EPAY cycle limit exceeded		
61	Account's ATM or EPAY cycle limit exceeded		
62	Bad card (on_us)		
65	Limit reached for total number of txns in cycle, independent of transaction category.		
68	Timer time out (used generally for issuers other than networks, i.e a host)		
75	Excessive pin failures		
76	Wrong Pin, Excessive pin failures		
77	The card has NOT ANY accounts		
78	Original transaction could not be found		

90	Response status unknown		
91	Service not available		
92	Invalid Payment Parameter		
93	Service blocked		
94	Duplicate transmission		
95	This can mean several things i.e. did not receive a tx amount being reversed greater than orig.		
96	Error (usually in pinblock translation)		
96	Cannot process transaction		
97	Service not allowed for client		
98	Invalid insurance number		
101	Service is already binded		
102	Service is not binded		
103	Invalid service data		
104	MAC error		
105	Debts absence.		
106	Invalid payment data		
107	Additional information required		
108	No such object in system		
109	Object is not created in system		
110	Object is already created in system		
111	Invalid CVV2		
112	Invalid Password		
113	Card is restricted		
0	The card is added to main cards!		

-100	Empty parameters!		
-101	Error while executing the request, try again!		
-102	Error while executing the request, try again!		
-103	Checkout required fields!		
-199	JsonRpc ERROR		
-200	Card not found!		
-201	Card expired, capture card!		
-202	Pan invalid, wrong format!		
-203	Card not re-issue!		
-204	Card status is not active!		
-205	Card blocked!		
-206	SMS is not active!		
-207	Transaction already exists in the database!		
-208	Transaction not found!		
-209	Wrong format!		
-210	Transfer between the same cards not possible!		
-211	Server is not responding, try again later!		
-212	Pan or expiry invalid, wrong format!		
-213	Card numbers are the same!		
-214	Invalid date! Date must not be less than or equal to the current!		
-215	Pan and phone not found!		
-216	Expiration date incorrect!		

-217	Wrong phone number format!		
-218	The length of the card number must be 6 digits!		
-219	The card has already entered to main cards!		
-220	The card is added to main cards!		
-221	The card is not main!		
-222	OLD and NEW main cards are not in the same account!		
-223	OLD_MAIN is not exist!		
-224	Phone numbers are not same!		
-225	NEW_MAIN is expired!		
-226	NEW_MAIN is not exist!		
-227	OLD_MAIN is not exist!		
-228	Limit not found!		
-229	Family card error!		
-230	Can not set limit to main card!		
-231	Main card error!		
-232	There are not main cards in this account!		
-233	There is other main card in this account!		
-234	This card is main!		
-235	Supported card not found!		
-236	Supported terminal not found!		
-237	Merchant not found!		
-238	Account not found!		
-239	Improper funds for holding!		

-240	Insufficient funds!		
-241	Wrong period for holding!		
-242	Hold not found!		
-243	Device not found!		
-244	Service is temporarily unavailable for this bank!		
-245	Amount does not fit!		
-246	Payment to this provider is prohibited!		
-247	EPOS does not exist!		
-248	Card valid!		
-249	Terminal Id and Merchant Id already exist in the database!		
-250	Wrong Terminal type!		
-251	Wrong format Terminal Id!		
-252	Wrong format Merchant Id!		
-253	Terminal Id and Merchant Id not found in SV!		
-254	Terminal Id and Merchant Id not found in the database!		
-255	Terminal Id and Merchant Id not registered!		
-256	You do not have access to the method!		
-257	Token not found!		
-258	Operation is not possible with corporate cards!		
-259	The amountCredit should not exceed the amount!		
-260	Insufficient funds - retry		

-261	Card was blocked in SVGate! Will be unblocked in		
-262	Allowed amount exceeded!		
-263	StatusId should be between 0-21!		
-264	MCC code not found!		
-265	UNKNOWN EXCEPTION IN PACKAGE		
-266	SMS is already activated!		
-267	Transaction not found in SV!		
-268	EPOS is not authorized!		
-269	OTP is not correct!		
-270	OTP has expired!		
-271	Token already exist!		
-272	You do not have access to the status!		
-273	You do not have access to receive card information!		
-274	You do not have access to change card status		
-275	The card does not belong to your bank, you cannot change the status of the card!		
-276	The card does not belong to your bank!		
-277	The number attached to the card and the send number do not match!		
-278	SMS informing is already active!		
-279	The card is not main!		
-280	Information not found!		

-281	Error while executing the request, try again!		
-282	Transaction not found or transaction was canceled!		
-283	ATTO PAY Exception!		
-284	ATTO Check Card Exception!		
-285	ATTO Card not found!		
-286	EPOS not found!		
-287	User not found!		
-288	Actions not found!		
-289	Card not blocked!		
-290	Phone invalid, wrong format!		
-291	Access denied!		
-292	Error sending SMS, try again later!		
-293	Daily pin reset limit used!		
-294	Request OTP not found!		
-295	You do not have access to a sms-notification activation!		
-296	The card does not belong to your bank, you cannot activate sms-notification!		
-297	Phone is not correct!		
-298	This is not an EPOS, this is a POS!		
-299	SMS message not found!		
-300	Service not available, try again later!		
-301	Check out the trans.ext method later!		

-302	Check out the trans.ext method later!		
-303	Service not available, try again later!		
-304	Service not available, try again later!		
-305	Service not available, try again later!		
-306	Service not available, try again later!		
-307	Time out SV!		
-308	Check out the trans.ext method later!		
-309	Check out the trans.ext method later!		
-310	Check out the trans.ext method later!		
-311	Check out the trans.ext method later!		
-314	Bad OTP request ID, try new OTP!		
-316	Terminal no access to payment methods!		
-318	Type terminal debit is not selected at the time of registration!		
-319	Contact the administrator to obtain permission!		
-320	The phone number attached to the card and the requesting phone number do not match!	Inform user to change the card or registration phone number	
-321	KEY is not correct!		
-322	KEY not found!		

-323	You do not have access to this limit!		
-324	LimitId is already set for this card!		
-325	Invalid date format!		
-326	LimitId is not set for this card!		
-327	The card does not belong to your bank, you can not set a limit for this card!		
-328	The card does not belong to your bank, you can not removed a limit for this card!		
-329	Wrong format parameter Date12!		
-330	Wrong format parameter Stan!		
-333	Errors from fraud monitoring(804! (UzKard))		
-357, ..., -350	Service not available, try again later (WCard)!		
-370	You do not have a monitoring contract!		
-371	Card already exists in the database!		
-372	History token not found!		
-373	Card not found in donate database!		
-374	EPOS not activated!		
-375	EPOS is already activated!		
-376	Phone number not entered!		
-377	Only tokens can be sent in this field!		
-378	Error while creating token, try again later!		

-379	You can send no more than 5 tokens!		
-380	Transaction successful, if you want to reverse send to trans.reverse method!		
-381	Exceeds the base amount!		
-382	BIN not found!		
-383	You cannot work with this issuer!		
-384	All fields are not filled!		
-385	Format Date of Birth is not correct!		
-386	Format document expiration date is not correct!		
-387	EPOS not authorized for p2p.universal method!		
-388	Merchant and credit card do not belong to the same institution!		
-389	Repeated reverse transaction!		
-390	Code not found!		
-392	Card does not belong to this PINFL!		
-393	PINFL not found!		
-398	Token created for replenishment only!		
-399	WCard errors!		
-604, ..., -400	Service not available, try again later!		

Humo error codes and their descriptions

001	Approved, honour with identification		
002	Approved for partial amount		

003	Approved for VIP		
004	Approved, update track 3		
005	Approved, account type specified by card issuer		
006	Approved for partial amount, account type specified by card issuer		
007	Approved, update ICC		
100	Decline (general, no comments)		
101	Decline, expired card		
102	Decline, suspected fraud		
103	Decline, card acceptor contact acquirer		
104	Decline, restricted card		
105	Decline, card acceptor call acquirer's security department		
106	Decline, allowable PIN tries exceeded		
107	Decline, refer to card issuer		
108	Decline, refer to card issuer's special conditions		
109	Decline, invalid merchant		
110	Decline, invalid amount		
111	Decline, invalid card number		
112	Decline, PIN data required		
113	Decline, unacceptable fee		
114	Decline, no account of type requested		
115	Decline, requested function not supported		

116	Decline, not sufficient funds		
117	Decline, incorrect PIN		
118	Decline, no card record		
119	Decline, transaction not permitted to cardholder		
120	Decline, transaction not permitted to terminal		
121	Decline, exceeds withdrawal amount limit		
122	Decline, security violation		
123	Decline, exceeds withdrawal frequency limit		
124	Decline, violation of law		
125	Decline, card not effective		
126	Decline, invalid PIN block		
127	Decline, PIN length error		
128	Decline, PIN key synch error		
129	Decline, suspected counterfeit card		
180	Decline, by cardholders wish		
200	Pick-up (general, no comments)		
201	Pick-up, expired card		
202	Pick-up, suspected fraud		
203	Pick-up, card acceptor contact card acquirer		
204	Pick-up, restricted card		
205	Pick-up, card acceptor call acquirer's security department		
206	Pick-up, allowable PIN tries exceeded		

207	Pick-up, special conditions		
208	Pick-up, lost card		
209	Pick-up, stolen card		
210	Pick-up, suspected counterfeit card		
300	Status message: file action successful		
301	Status message: file action not supported by receiver		
302	Status message: unable to locate record on file		
303	Status message: duplicate record, old record replaced		
304	Status message: file record field edit error		
305	Status message: file locked out		
306	Status message: file action not successful		
307	Status message: file data format error		
308	Status message: duplicate record, new record rejected		
309	Status message: unknown file		
400	Accepted (for reversal)		
401	Mobile Banking hasn't been activated on given card by given phone number	Inform user to change the card or registration phone number	
499	Approved, no original message data		
500	Status message: reconciled, in balance		
501	Status message: reconciled, out of balance		

502	Status message: amount not reconciled, totals provided		
503	Status message: totals for reconciliation not available		
504	Status message: not reconciled, totals provided		
600	Accepted (for administrative info)		
601	Status message: impossible to trace back original transaction		
602	Status message: invalid transaction reference number		
603	Status message: reference number/PAN incompatible		
604	Status message: POS photograph is not available		
605	Status message: requested item supplied		
606	Status message: request cannot be fulfilled - required documentation is not available		
680	List ready		
681	List not ready		
700	Accepted (for fee collection)		
800	Accepted (for network management)		
900	Advice acknowledged, no financial liability accepted		
901	Advice acknowledged, financial liability accepted		
902	Decline reason message: invalid transaction		
903	Status message: re-enter transaction		

904	Decline reason message: format error		
905	Decline reason message: acquirer not supported by switch		
906	Decline reason message: cutover in process		
907	Decline reason message: card issuer or switch inoperative		
908	Decline reason message: transaction destination cannot be found for routing		
909	Decline reason message: system malfunction		
910	Decline reason message: card issuer signed off		
911	Decline reason message: card issuer timed out		
912	Decline reason message: card issuer unavailable		
913	Decline reason message: duplicate transmission		
914	Decline reason message: not able to trace back to original transaction		
915	Decline reason message: reconciliation cutover or checkpoint error		
916	Decline reason message: MAC incorrect		
917	Decline reason message: MAC key sync error		
918	Decline reason message: no communication keys available for use		
919	Decline reason message: encryption key sync error		

920	Decline reason message: security software/hardware error - try again		

Tajikistan error codes and their description

32606	The card number is not valid or supported		
4012	The entered amount is outside the allowed range.		
4011	Sum too few		
4010	The application was rejected by the provider		
4008	Payment in the process. Cancel payment isn't provided		
4006	No response from provider. Please try again later.		
4004	Transaction ID not found		
4001	The service is temporarily unavailable.		
4000	Transaction id already exists		

Kyrgyzstan error codes and their description

-32101	Username or password incorrect		
--------	--------------------------------	--	--

-32103	Authorization token expired or not valid		
-32104	Connect timeout error occurred		
-32105	Not allowed method request		
-32401	External ID already exists!		
-32402	External ID not found..		
-32409	Partner is not active		
-32410	Amount is not allowed		
-32411	Commission calculation failed		
-32412	Min Amount-related error		
-32413	Max amount-related error		
-32700	Request body incorrect		
-32701	JsonRPC values format is not valid		